



BITS Pilani
Pilani Campus

Unsupervised Deep Learning ZG513

Dr. Sugata Ghosal
sugata.ghosal@pilani.bits-pilani.ac.in



Session 4

Likelihood-Based Autoregressive Models

These slides are prepared by the instructor, with grateful acknowledgement of many others who made their course materials freely available online.

Lecture overview



- **Likelihood Based Models**
 - Simple generative models: histograms
 - Parameterized distributions and maximum likelihood
- **Autoregressive Models**
 - Recurrent Neural Nets
 - Masking-based Models
 - Masked Autoencoder for Density Estimation (MADE)
 - Wavenet
 - PixelCNN and Variants

Likelihood-based models

- Problems we'd like to solve:
 - Generating data: synthesizing images, videos, speech, text
 - Compressing data: constructing efficient codes
 - Anomaly detection
- Likelihood-based models
 - Estimate p_{data} from samples $x^{(1)}, \dots, x^{(n)} \sim p_{\text{data}}(x)$
- Learns a distribution p that allows:
 - Computing $p(x)$ for arbitrary x
 - Sampling $x \sim p(x)$

Desiderata

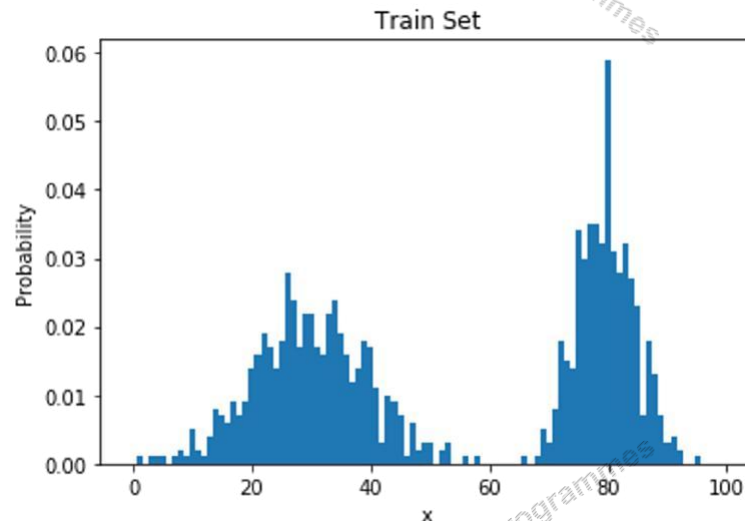


- We want to estimate distributions of **complex, high-dimensional data**
 - A $128 \times 128 \times 3$ image lies in a $\sim 50,000$ -dimensional space
- We also want computational and statistical efficiency
 - Efficient training and model representation
 - Expressiveness and generalization
 - Sampling quality and speed
 - Compression rate and speed

Learning: Estimate frequencies by counting



- Recall: the goal is to estimate p_{data} from samples $x^{(1)}, \dots, x^{(n)} \sim p_{\text{data}}(x)$
- Suppose the samples take on values in a finite set $\{1, \dots, k\}$
- The model: a histogram
 - (Redundantly) described by k nonnegative numbers: $p_1, \dots, p_k$
 - To train this model: count frequencies p_i
= (# times i appears in the dataset) /
(# points in the dataset)
- How can we compute likelihoods and sample from this model?



Inference and Sampling



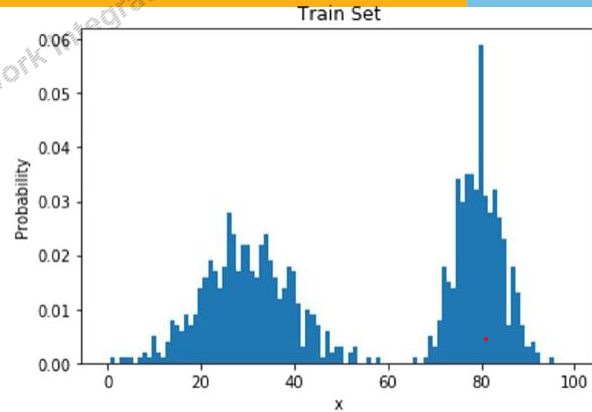
- **Inference** (querying p_i for arbitrary i): simply a lookup into the array $p_1, \dots, p_k$
- **Sampling** (lookup into the inverse cumulative distribution function)
 1. From the model probabilities $p_1, \dots, p_k$, compute the cumulative distribution
$$F_i = p_1 + \dots + p_i \quad \text{for all } i \in \{1, \dots, k\}$$
 2. Draw a uniform random number $u \sim [0, 1]$
 3. Return the smallest i such that $u \leq F_i$
- **Are we done?**

Failure in high dimensions



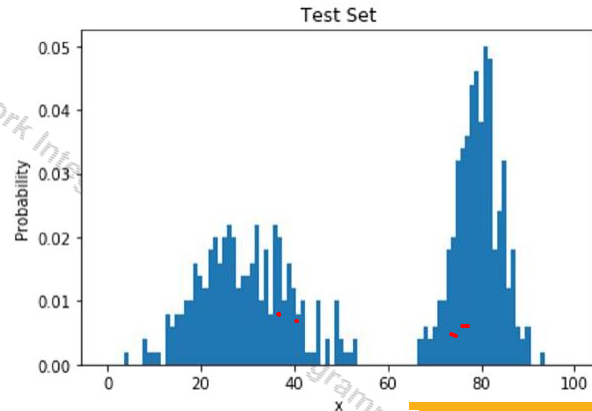
- No, because of the **curse of dimensionality**. Counting fails when there are too many bins.
 - (Binary) MNIST: 28x28 images, each pixel in {0, 1}
 - There are $2^{784} \approx 10^{236}$ probabilities to estimate
 - Any reasonable training set covers only a tiny fraction of this
 - No generalization whatsoever! There is shared structure among images but this kind of model would not be able to capture that.

Problematic even for single variable



learned histogram = training data distribution

→ often poor generalization



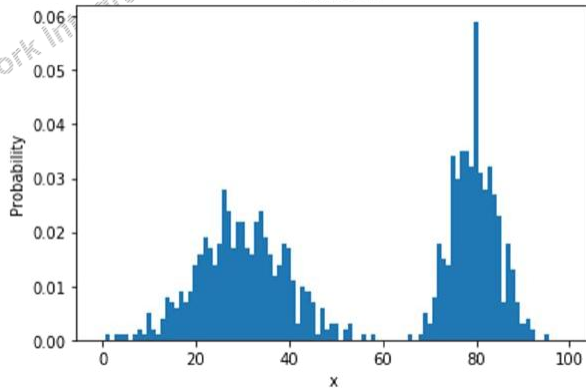
Parameterized Distributions

innovate

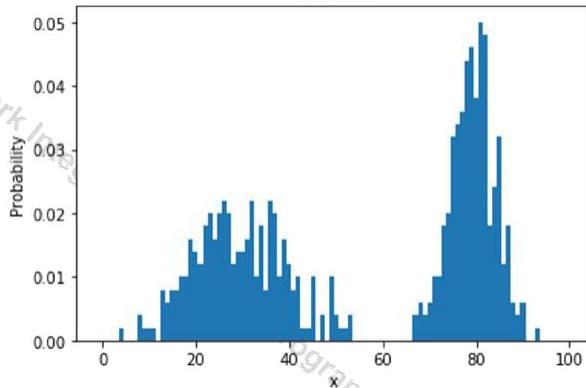
achieve

lead

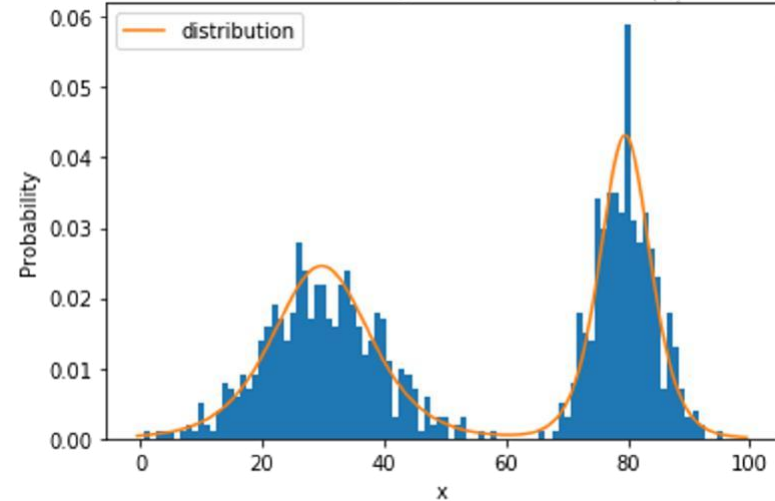
Train Set



Test Set



Epoch 99



Fitting a parameterized distribution
often generalizes better

Likelihood-based generative models

- Recall: the goal is to estimate p_{data} from $x^{(1)}, \dots, x^{(n)} \sim p_{\text{data}}(x)$
- Now we introduce **function approximation**: learn θ so that $p_{\theta}(x) \approx p_{\text{data}}(x)$.
 - How do we design easy-to-train function approximators to effectively represent complex joint distributions over x ?
 - There will be many choices for model design, each with different tradeoffs and different compatibility criteria.
- **Designing the model and the training procedure go hand-in-hand.**

Fitting distributions

- Given data $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(n)}$ sampled from a “true” distribution p_{data}
- Set up a model class: a set of parameterized distributions p_{θ}
- Formulate a search problem over parameters

$$\arg \min_{\theta} \text{loss}(\theta, \mathbf{x}^{(1)}, \dots, \mathbf{x}^{(n)})$$

- Want the loss function + search procedure to:
 - Work with large datasets (n is large, say millions of training examples)
 - Yield θ such that p_{θ} matches p_{data} — i.e. the training algorithm *works*. Think of the loss as a distance between distributions.
 - Note that the training procedure can only see the empirical data distribution, not the true data distribution: we want the model to generalize.



Maximum likelihood

- Maximum likelihood: given a dataset $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(n)}$, find θ by solving the optimization problem

$$\arg \min_{\theta} \text{loss}(\theta, \mathbf{x}^{(1)}, \dots, \mathbf{x}^{(n)}) = \frac{1}{n} \sum_{i=1}^n -\log p_{\theta}(\mathbf{x}^{(i)})$$

- Statistics tells us that if the model family is expressive enough and if enough data is given, then solving the maximum likelihood problem will yield parameters that generate the data
- Equivalent to minimizing KL divergence between the empirical data distribution and the model

$$\hat{p}_{\text{data}}(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}[\mathbf{x} = \mathbf{x}^{(i)}]$$

$$\text{KL}(\hat{p}_{\text{data}} \parallel p_{\theta}) = \mathbb{E}_{\mathbf{x} \sim \hat{p}_{\text{data}}} [-\log p_{\theta}(\mathbf{x})] - H(\hat{p}_{\text{data}})$$



Stochastic gradient descent

- Maximum likelihood is an optimization problem. How do we solve it?
- **Stochastic gradient descent (SGD).**
 - SGD minimizes expectations: for f a differentiable function of θ , it solves

$$\arg \min_{\theta} \mathbb{E}[f(\theta)]$$

- With maximum likelihood, the optimization problem is

$$\arg \min_{\theta} \mathbb{E}_{\mathbf{x} \sim \hat{p}_{\text{data}}} [-\log p_{\theta}(\mathbf{x})]$$

- **Why maximum likelihood + SGD?** It works with large datasets and is compatible with neural networks.



Designing the model



- Key requirement for maximum likelihood + SGD: efficiently compute $\log p(\mathbf{x})$ and its gradient
- We will choose models p_θ to be deep neural networks, which work in the regime of high expressiveness and efficient computation (assuming specialized hardware)
- How exactly do we design these networks?

- Any setting of θ must define a valid probability distribution over $\mathbf{x}$:

$$\text{for all } \theta, \quad \sum_{\mathbf{x}} p_\theta(\mathbf{x}) = 1 \quad \text{and} \quad p_\theta(\mathbf{x}) \geq 0 \quad \text{for all } \mathbf{x}$$

- $\log p_\theta(\mathbf{x})$ should be easy to evaluate and differentiate with respect to θ
- This can be tricky to set up!

Autoregressive models



- First, given a Bayes net structure, setting the conditional distributions to neural networks will yield a tractable log likelihood and gradient. Great for maximum likelihood training!

$$\log p_{\theta}(\mathbf{x}) = \sum_{i=1}^d \log p_{\theta}(x_i \mid \text{parents}(x_i))$$

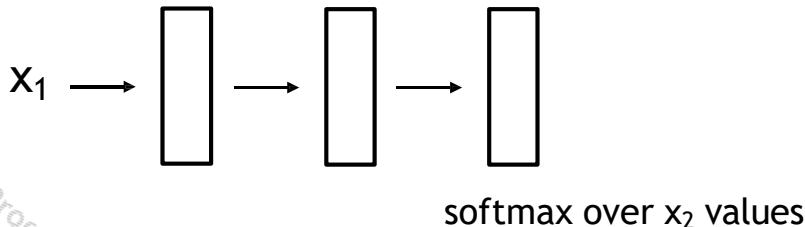
- But is it expressive enough? Yes, assuming a fully expressive Bayes net structure: any joint distribution can be written as a product of conditionals

$$\log p(\mathbf{x}) = \sum_{i=1}^d \log p(x_i \mid \mathbf{x}_{1:i-1})$$

- This is called an **autoregressive model**. So, an expressive Bayes net structure with neural network conditional distributions yields an expressive model for $p(\mathbf{x})$ with tractable maximum likelihood training.

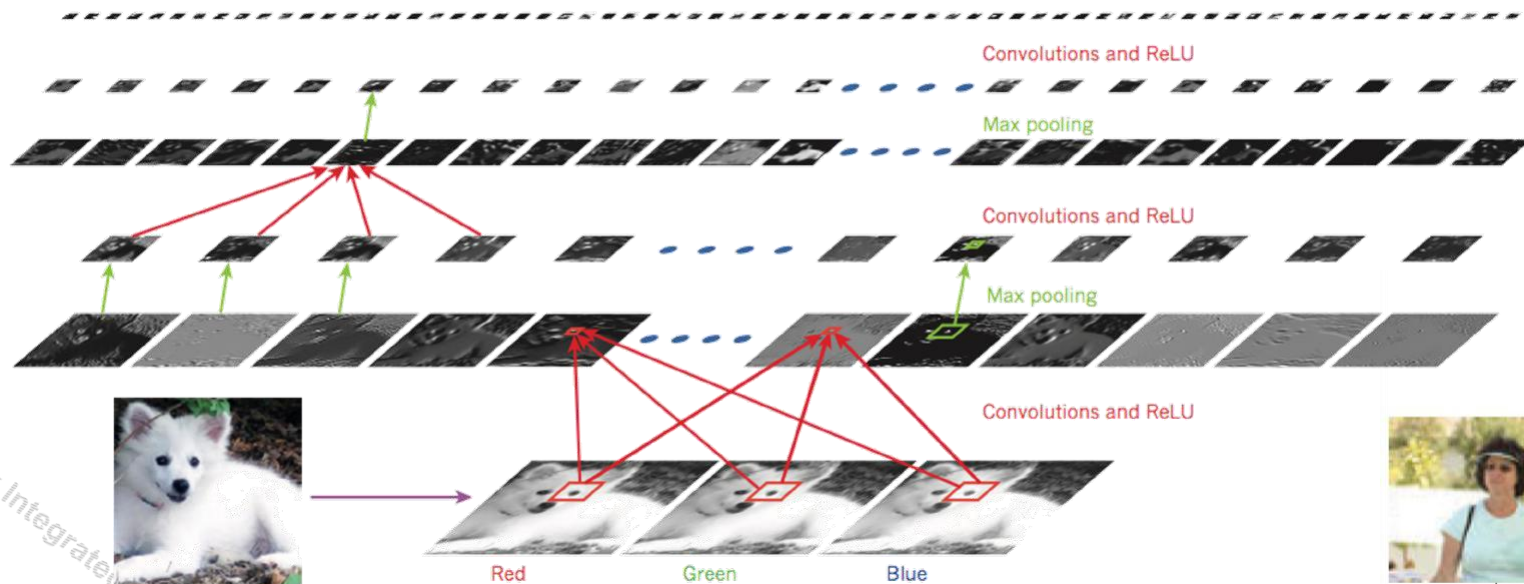
A toy autoregressive model

- Two variables: x_1, x_2
- Model: $p(x_1, x_2) = p(x_1) p(x_2 | x_1)$
 - $p(x_1)$ is a histogram
 - $p(x_2 | x_1)$ is a multilayer perceptron
 - Input is x_1
 - Output is a distribution over x_2 (logits, followed by softmax)



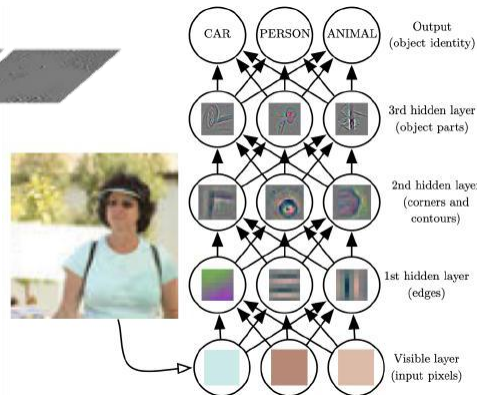
Neural building blocks: CNNs

Samoyed (16); Papillon (5.7); Pomeranian (2.7); Arctic fox (1.0); Eskimo dog (0.6); white wolf (0.4); Siberian husky (0.4)

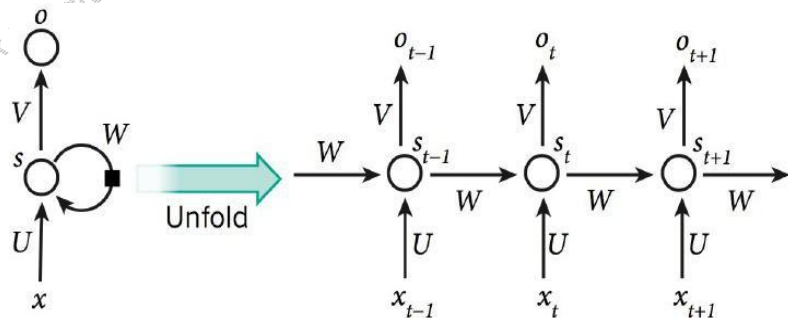


A Convolutional Neural Network (CNN)

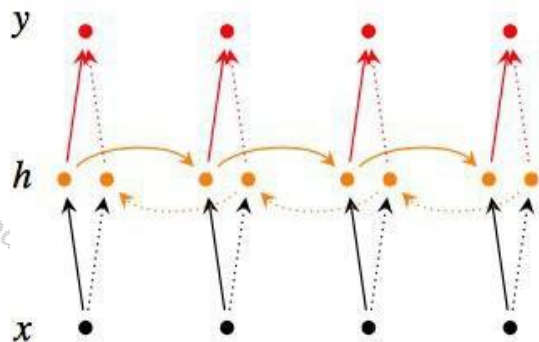
Y. LeCun, Y. Bengio, G. Hinton, "Deep Learning", Nature, Vol. 521, 28 May 2015



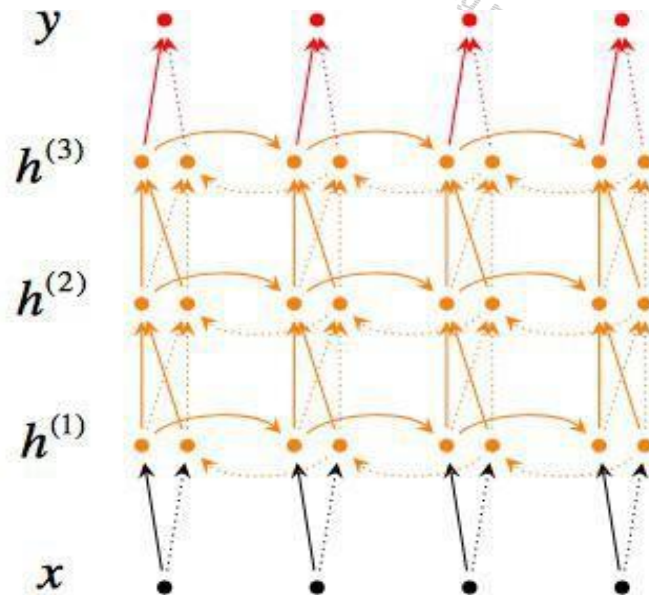
Neural building blocks: RNNs



A Recurrent Neural Network (RNN)
(unfolded across time-steps)



A bi-directional RNN



A deep bi-directional RNN

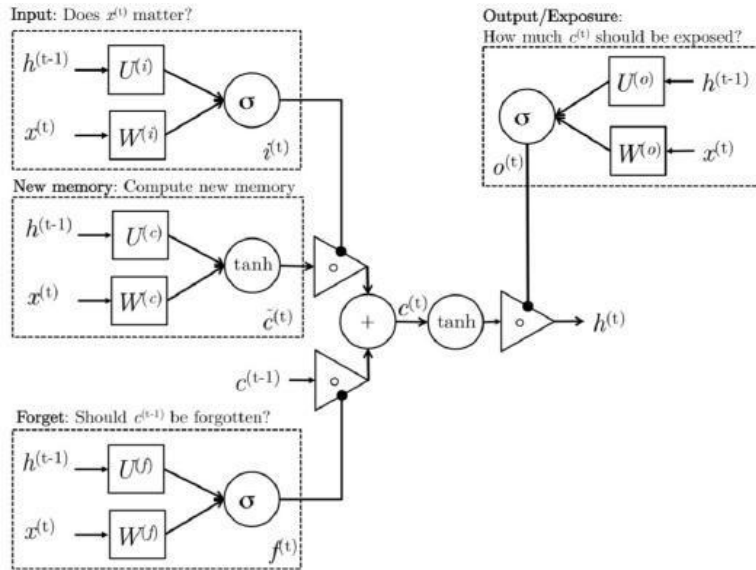
C. Manning and R Socher, Stanford CS224n Lecture 8 Notes
Y. LeCun, Y. Bengio, G. Hinton, "Deep Learning", Nature, Vol. 521, 28 May 2015

Neural building blocks: RNNs

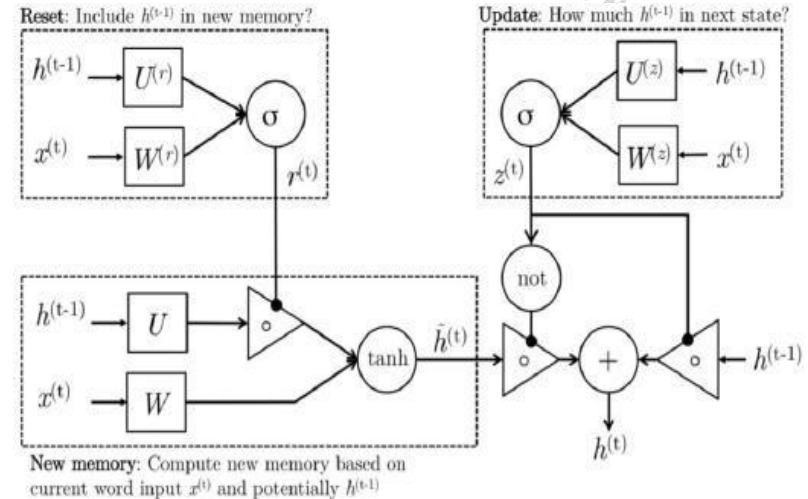
innovate

achieve

lead



Long-Short-Term-Memories (LSTMs)



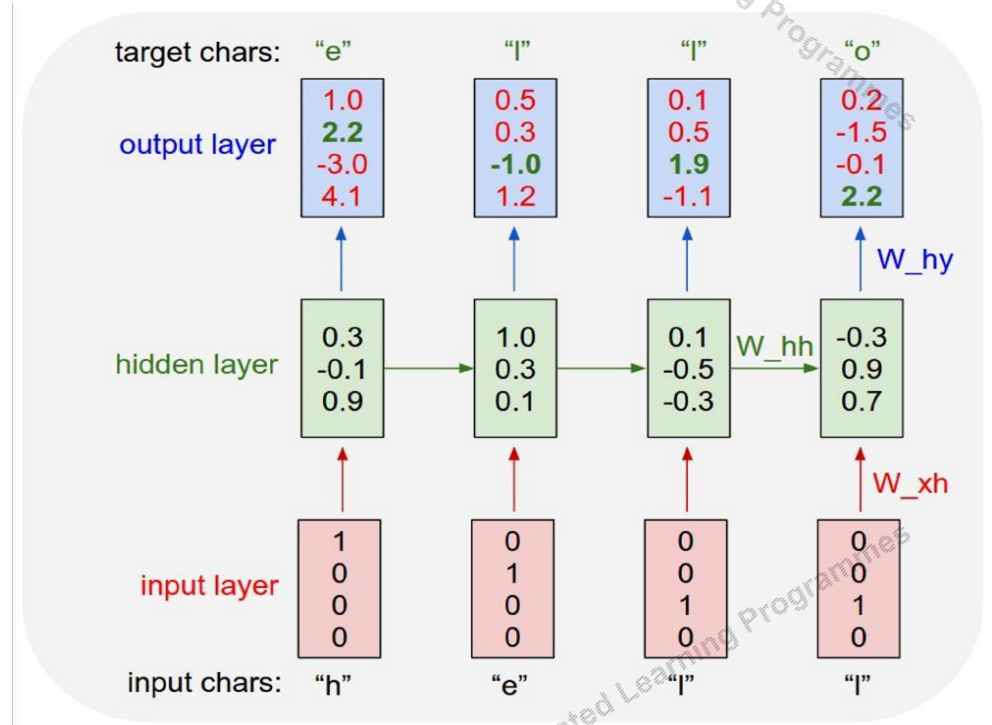
Gated Recurrent Units (GRUs)

RNN models - char-RNN

$$\log p(\mathbf{x}) = \sum_{i=1}^d \log p(x_i | \mathbf{x}_{1:i-1})$$

Sequence of
characters

Character
at i^{th}
position



MNIST

- Handwritten digits
- 28x28
- 60,000 train
- 10,000 test
- Original: greyscale
- “Binarized MNIST” -- 0/1 (black/white)



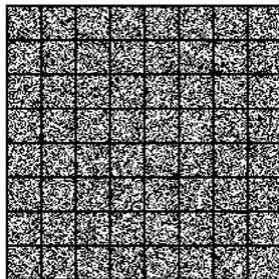
RNN on MNIST

innovate

achieve

lead

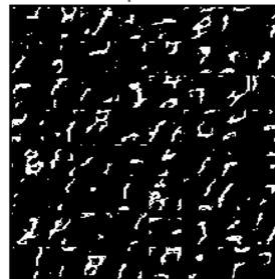
Initialization



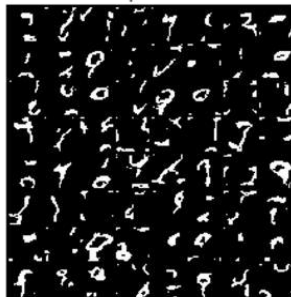
Epoch 0



Epoch 1



Epoch 2



Epoch 8

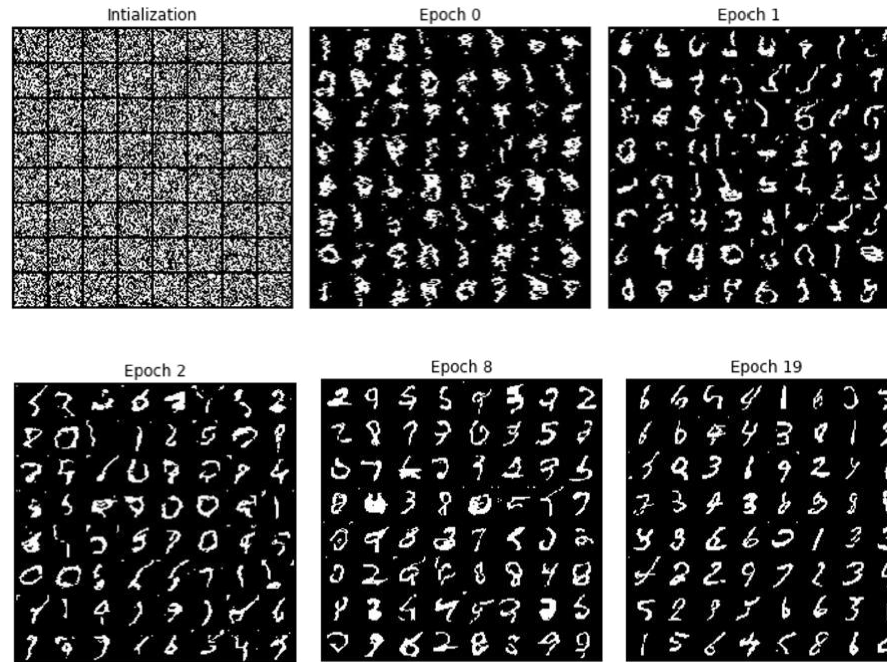


Epoch 19



RNN with Pixel Location Appended on MNIST

- Append (x,y) coordinates of pixel in the image as input to RNN



Masking-based autoregressive models



- Second major branch of neural AR models. Key property is parallelized computation of all conditionals
 - Masked MLP (MADE)
 - Masked convolutions & self-attention
 - Also share parameters across time

<https://neuroverse0.wordpress.com/2020/07/14/made-masked-autoencoder-for-distribution-estimation/>

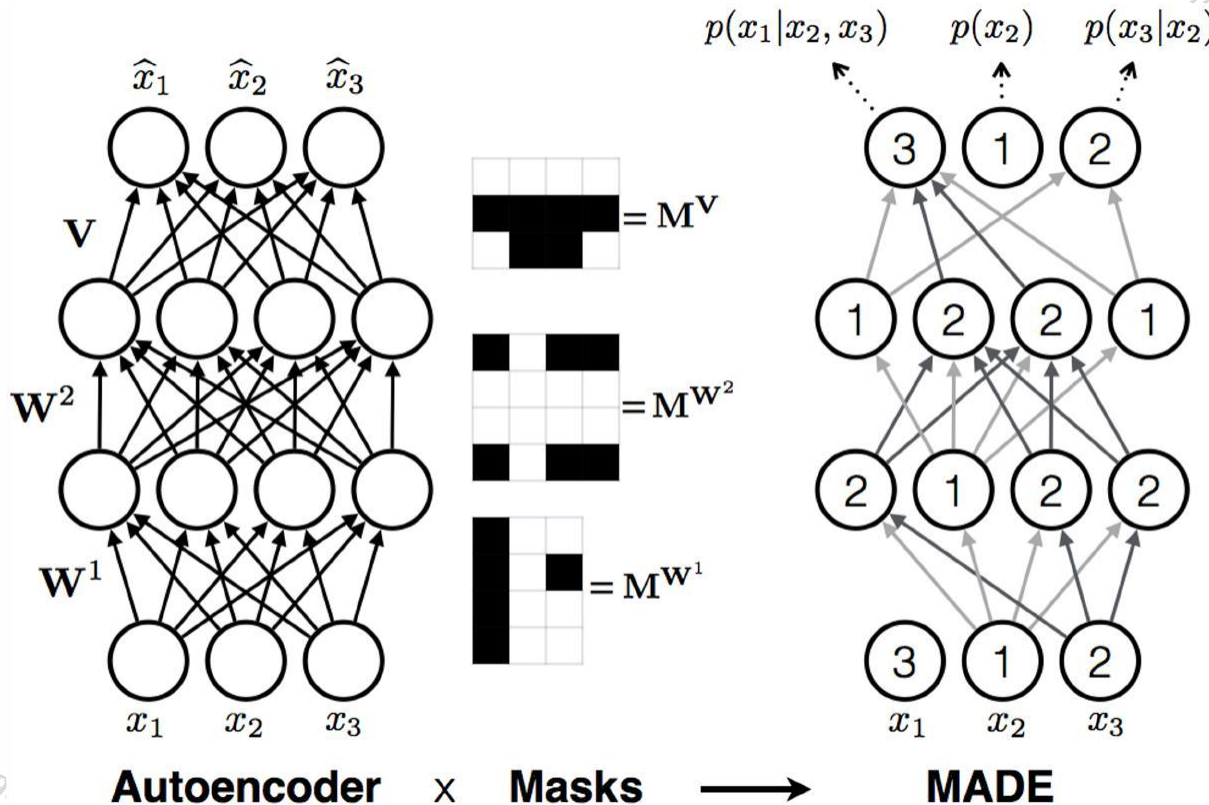
<https://nsanghi.com/blog/made/>

Masked AE for Distribution Estimation (MADE)

innovate

achieve

lead

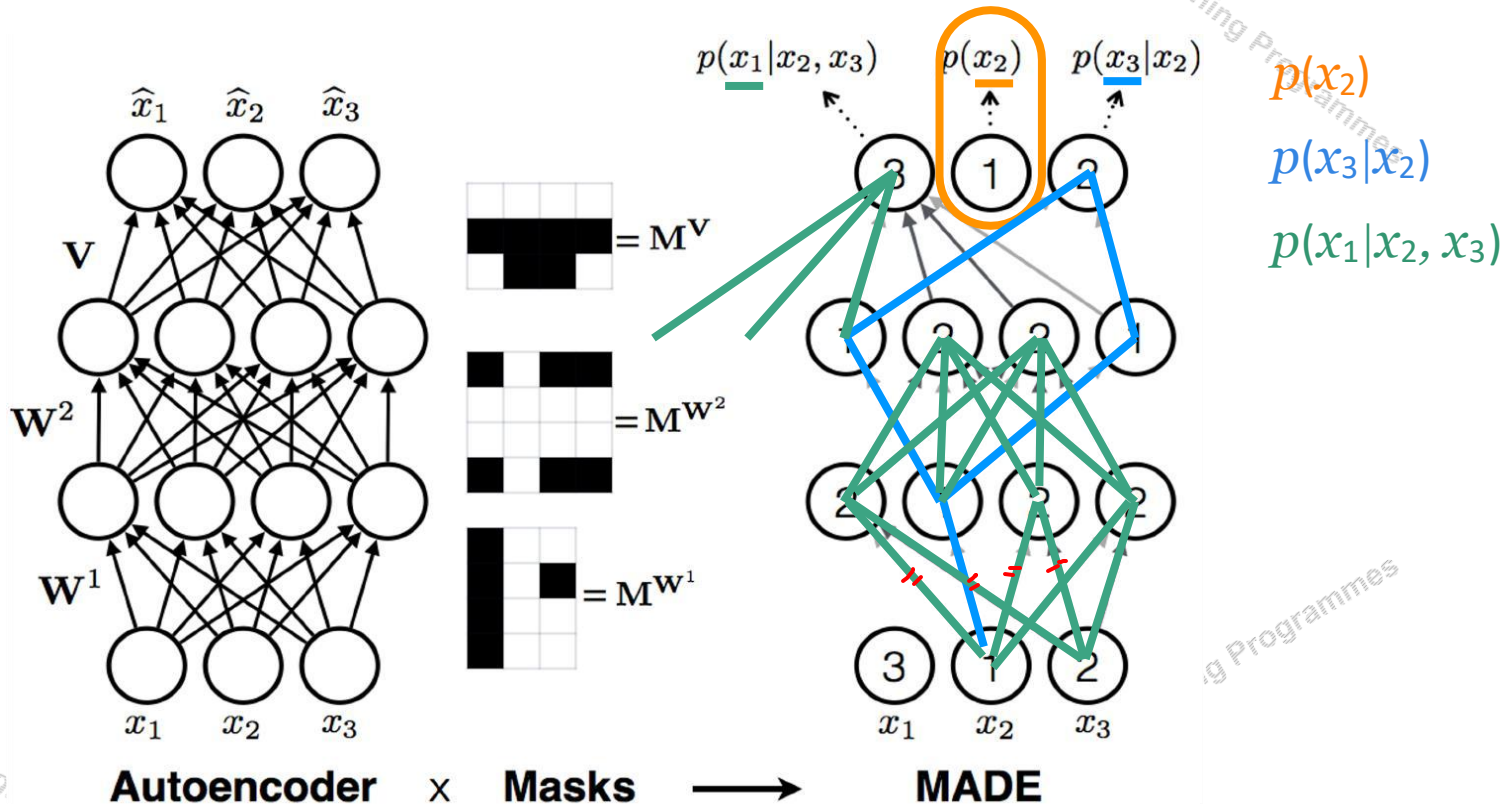


Masked AE for Distribution Estimation (MADE)

innovate

achieve

lead



Masked AE for Distribution Estimation

innovate

achieve

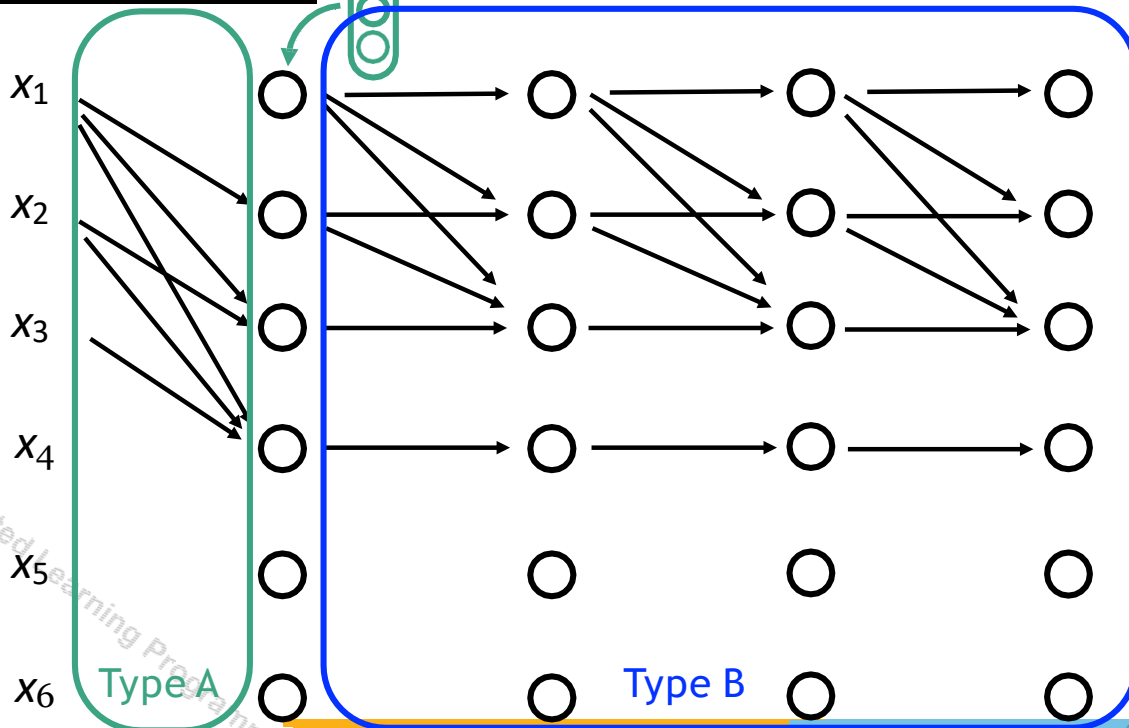
lead

$$p(x) = p(x_1) \cdot p(x_2|x_1) \cdots p(x_6|x_{1:5})$$

$$\max_{\theta} \sum_{i=1}^N \sum_k \log p_{\theta}(x_k^{(i)} | x_{1:k-1}^{(i)})$$

Sampling is slow!

General Principle



$p(x_1)$

$p(x_2|x_1)$

$p(x_3|x_{1:2})$

$p(x_4|x_{1:3})$

$p(x_5|x_{1:4})$

$p(x_6|x_{1:5})$

Sampling



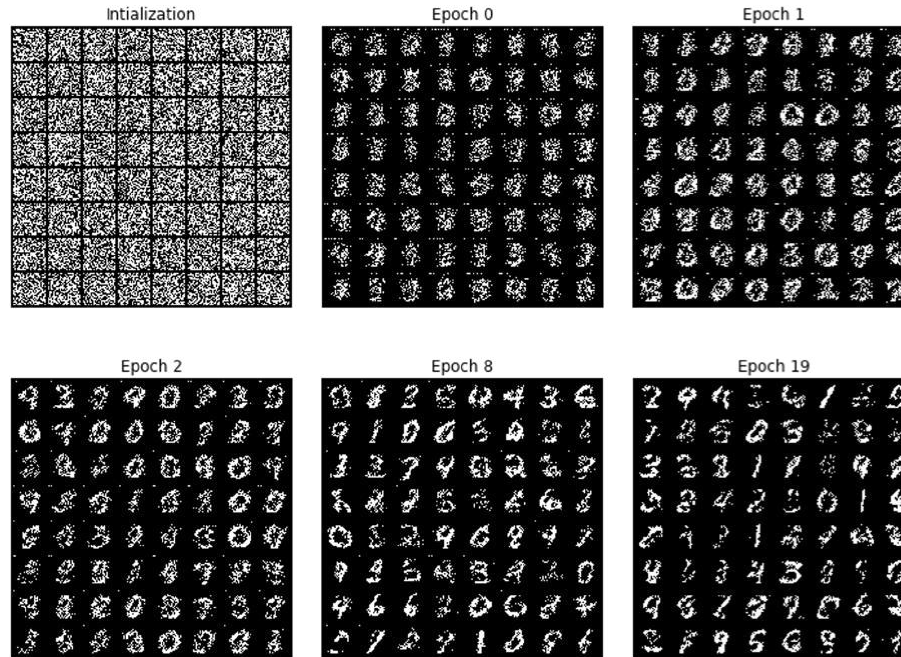
- Randomly generate vector x , set $i = 1$
- Feed x into autoencoder and generate outputs $\hat{x}$ for the network, set $p = \hat{x}_i$.
- Sample from a Bernoulli distribution with parameter p , set input $x_i = \text{Bernoulli}(p)$.
- Increment i and repeat steps 2-4 until $i > D$.

MADE on MNIST

innovate

achieve

lead



MADE results

innovate

achieve

lead

Negative log-likelihood test results of different models on the binarized MNIST dataset.

Model	$-\log p$
MoBernoullis K=10	168.95
MoBernoullis K=500	137.64
MADE 1hl (1 mask)	88.40
MADE 2hl (1 mask)	89.59
MADE 1hl (32 masks)	88.04
MADE 2hl (32 masks)	86.64

Tractable

MADE results

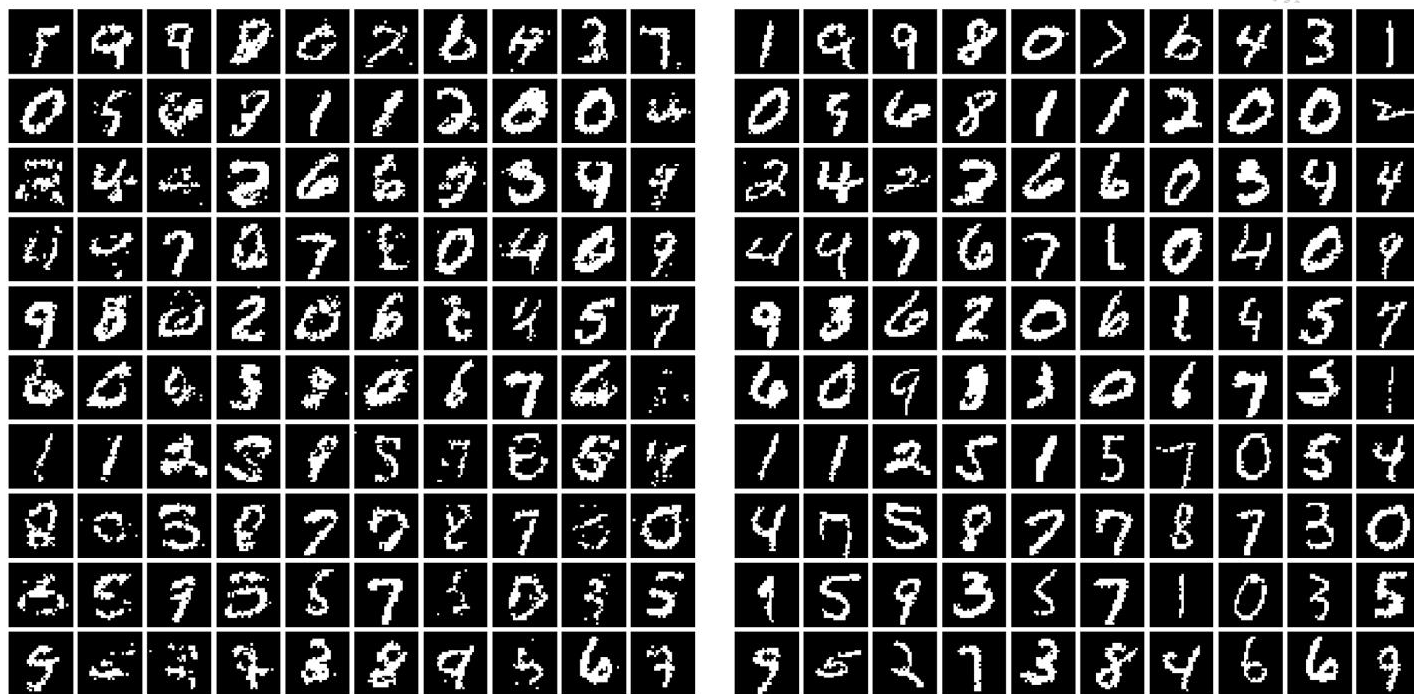


Figure 3. Left: Samples from a 2 hidden layer MADE. Right: Nearest neighbour in binarized MNIST.

MADE: Different Orderings

innovate

achieve

lead

- All orderings achieve roughly the same NLL but samples are different

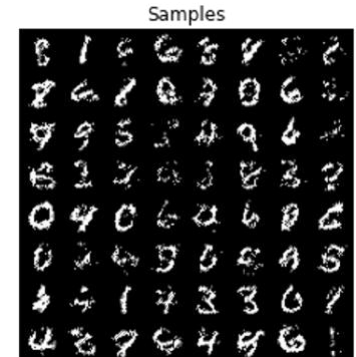
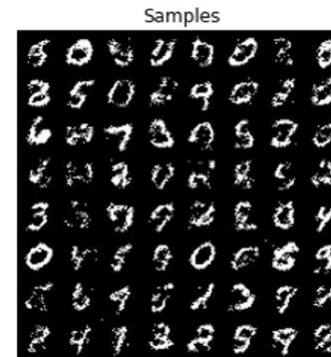
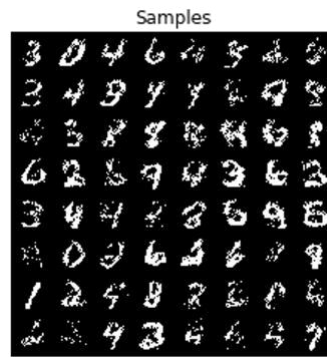
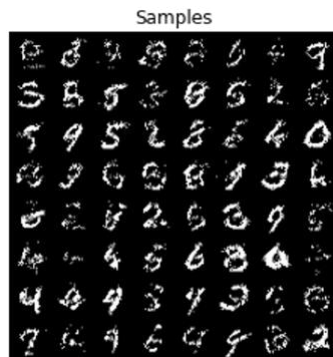
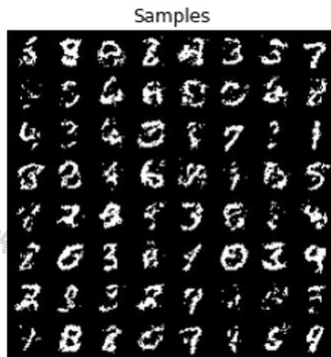
Random
Permutation

Even then Odd
Indices

Rows
(Raster Scan)

Columns

Top to Middle,
Bottom to Middle

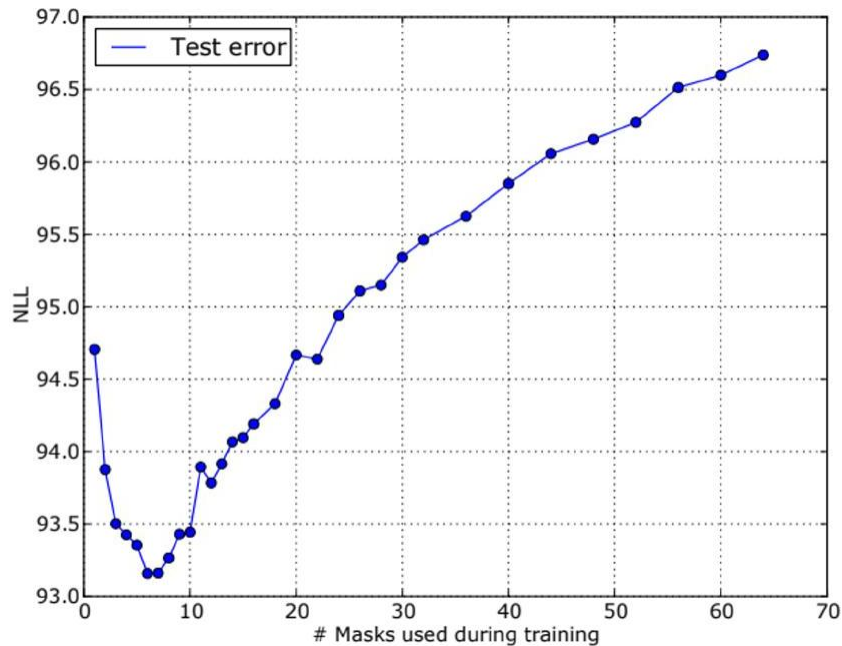


MADE: Multiple Orderings

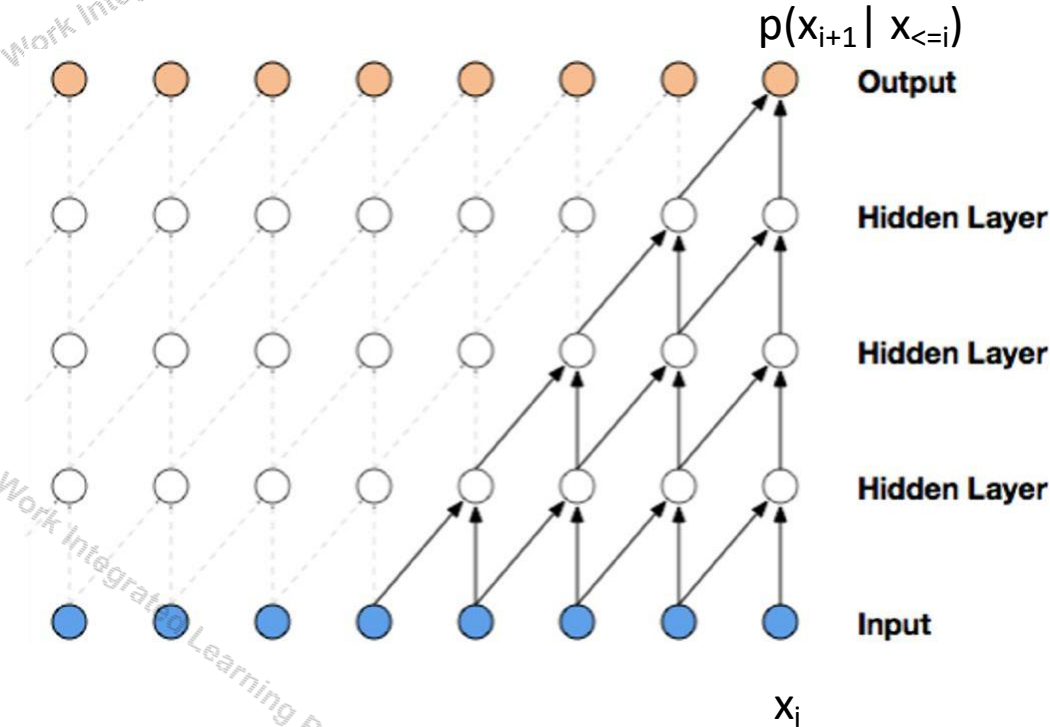
innovate

achieve

lead



Masked Temporal (1D) Convolution



- Easy to implement, masking part of the conv kernel
- Constant parameter count for variable-length distribution!
- Efficient to compute, convolution has hyper-optimized implementations on all hardware

However

- Limited receptive field, linear in number of layers

WaveNet – Causal Dilated Convolution



Input

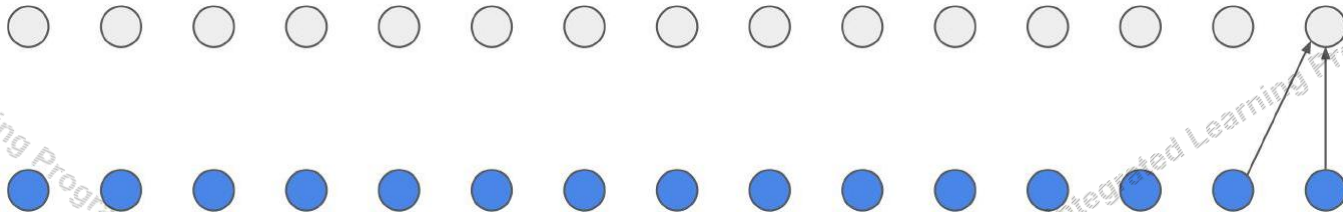


WaveNet – Causal Dilated Convolution

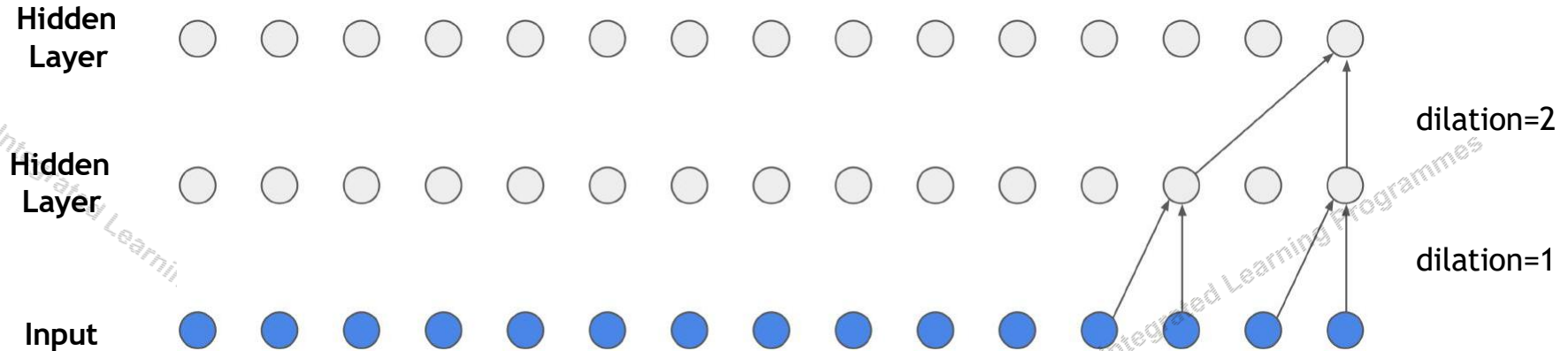


Hidden
Layer

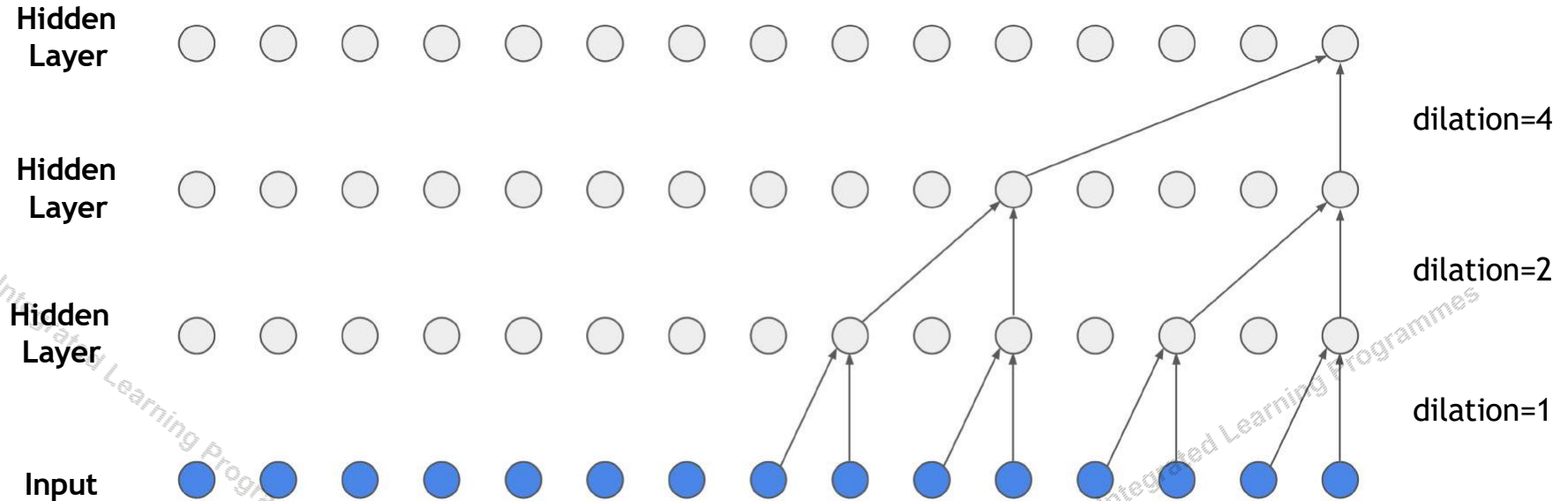
Input



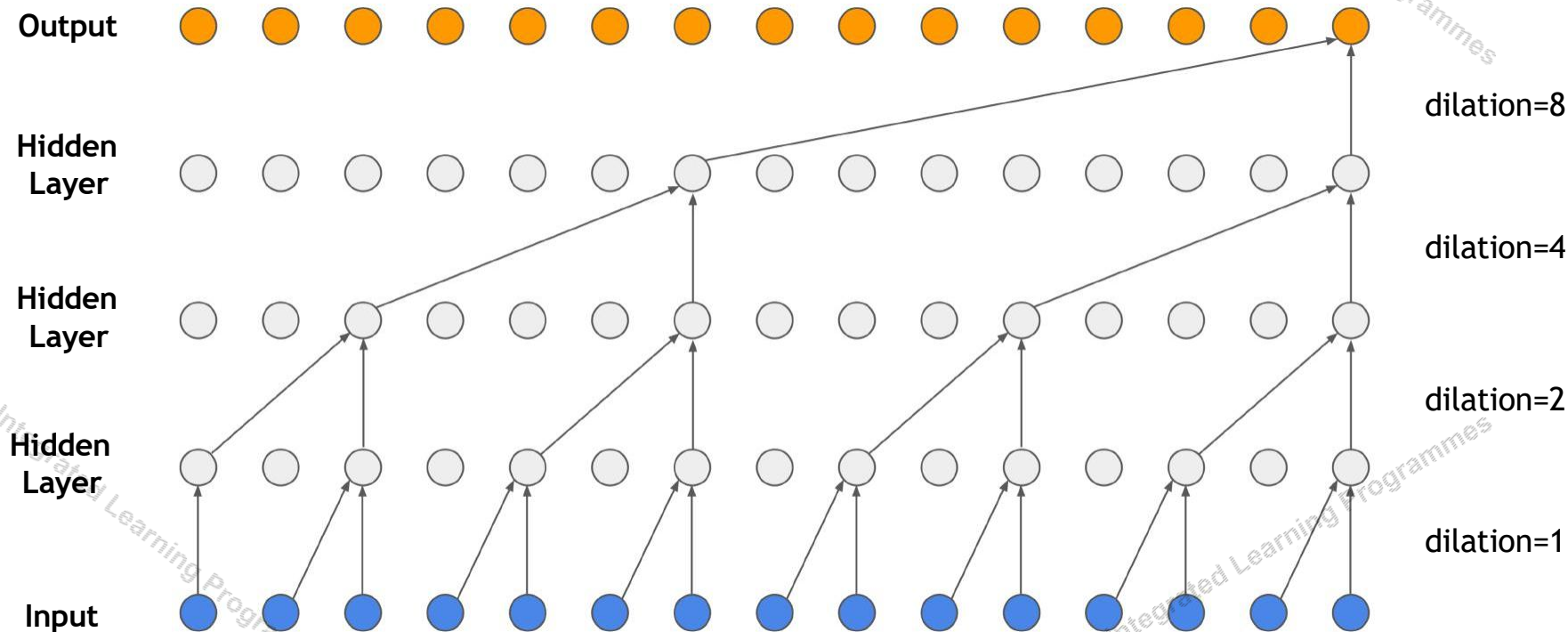
WaveNet – Causal Dilated Convolution



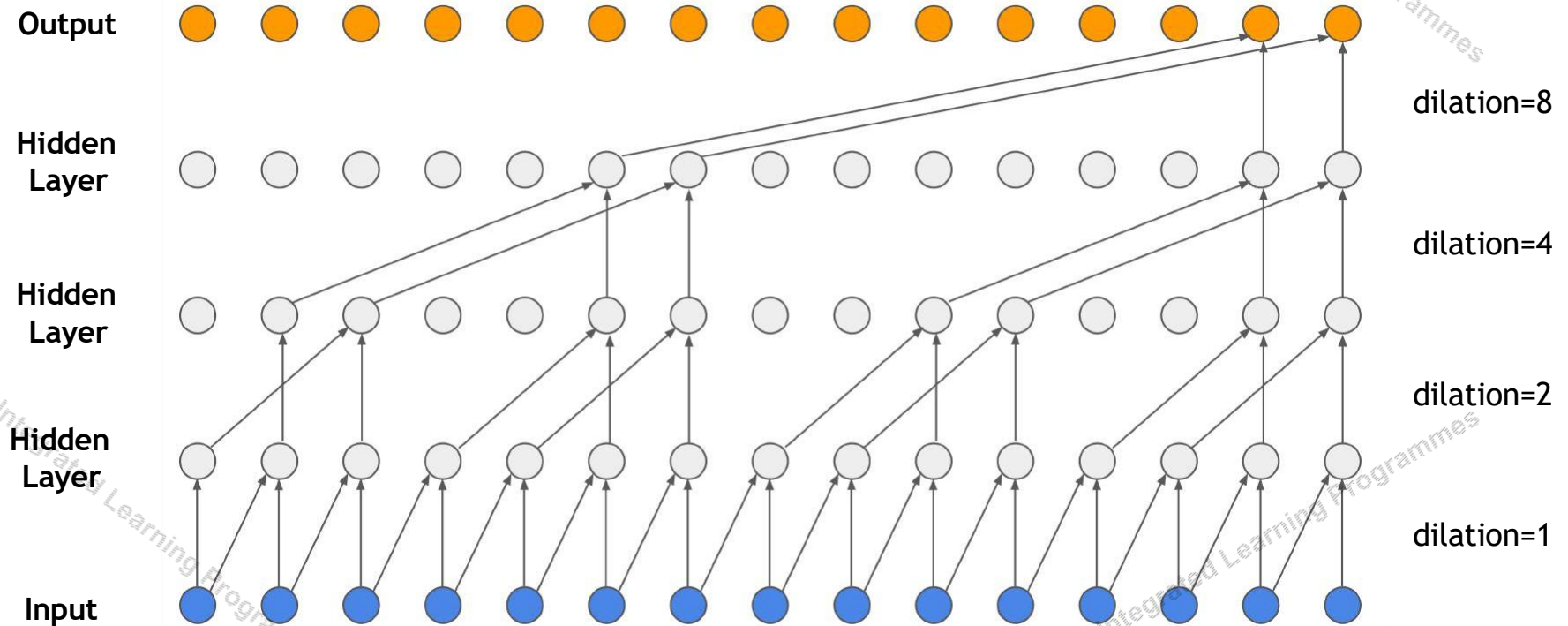
WaveNet – Causal Dilated Convolution



WaveNet – Causal Dilated Convolution

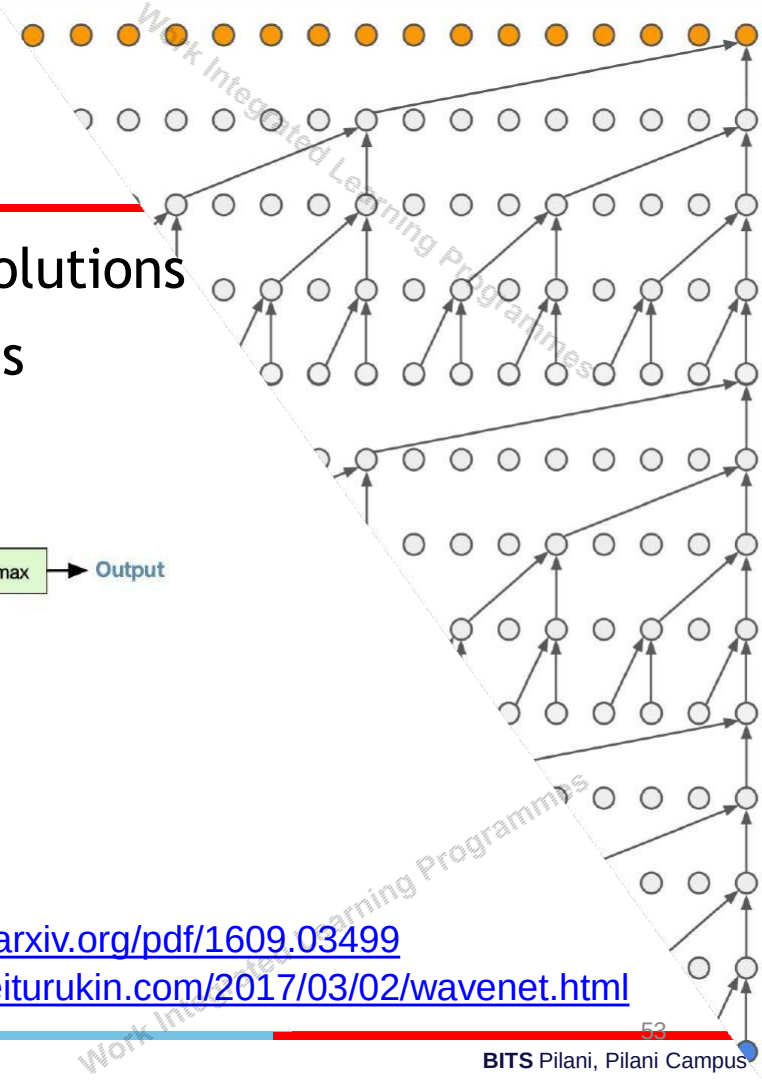
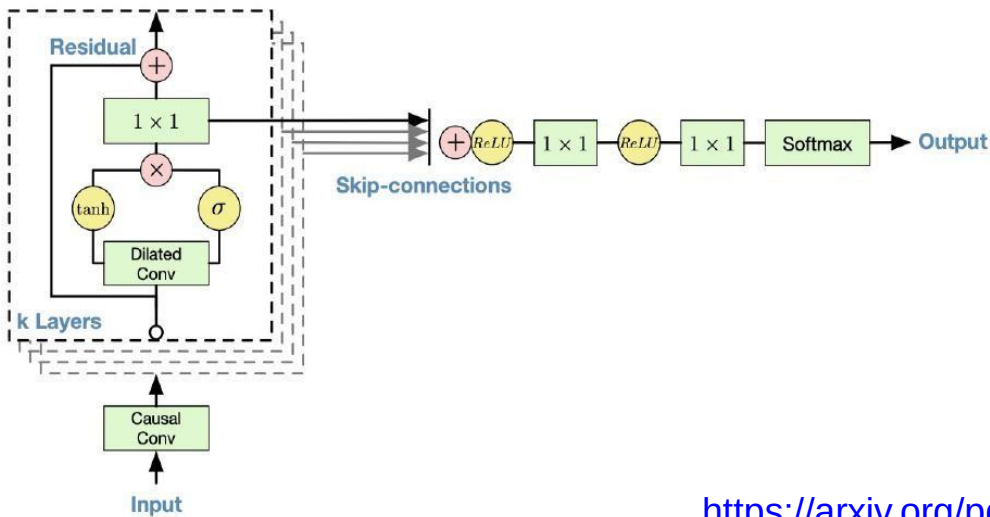


WaveNet – Causal Dilated Convolution



WaveNet – Multiple Stacks

- Improved receptive field with dilated convolutions
- Gated Residual block with skip connections

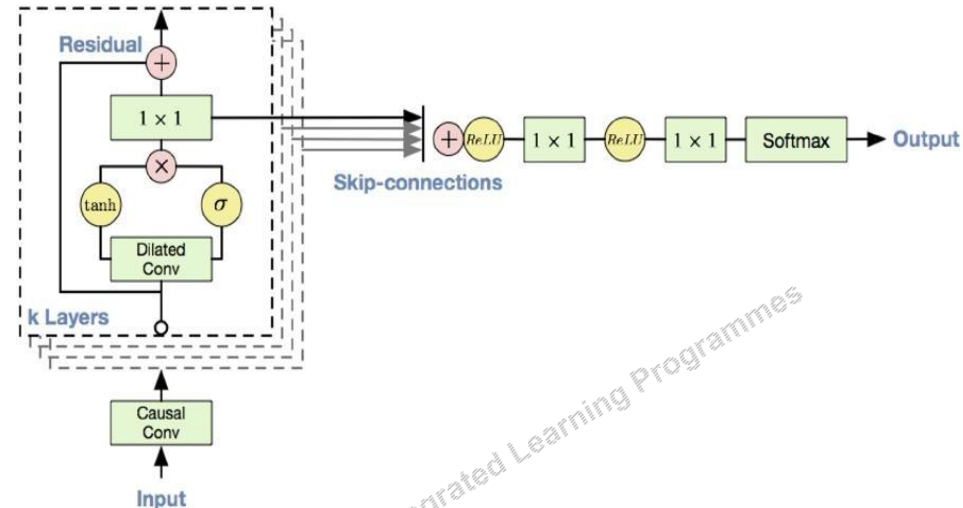


<https://arxiv.org/pdf/1609.03499>

<https://sergeituruikin.com/2017/03/02/wavenet.html>

WaveNet

- Improved receptive field: dilated convolution, with exponential dilation
- Better expressivity: Gated Residual blocks with Skip connections



<https://arxiv.org/pdf/1609.03499>

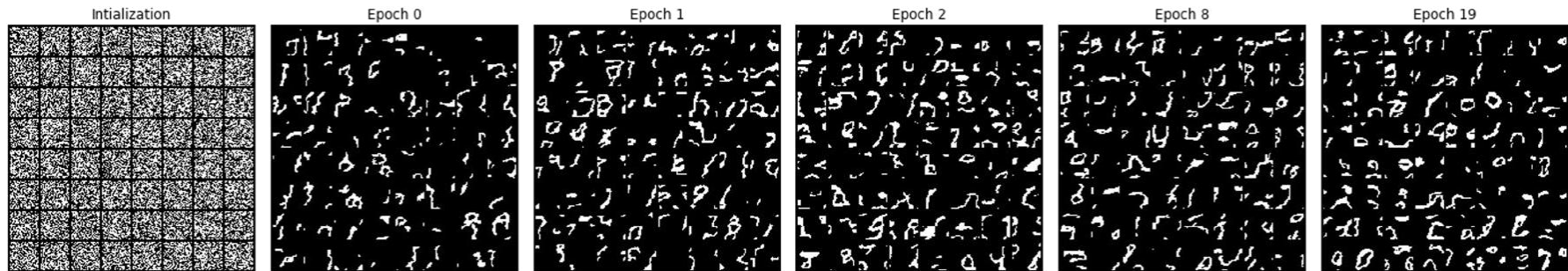
<https://sergeiturukin.com/2017/03/02/wavenet.html>

WaveNet on MNIST

innovate

achieve

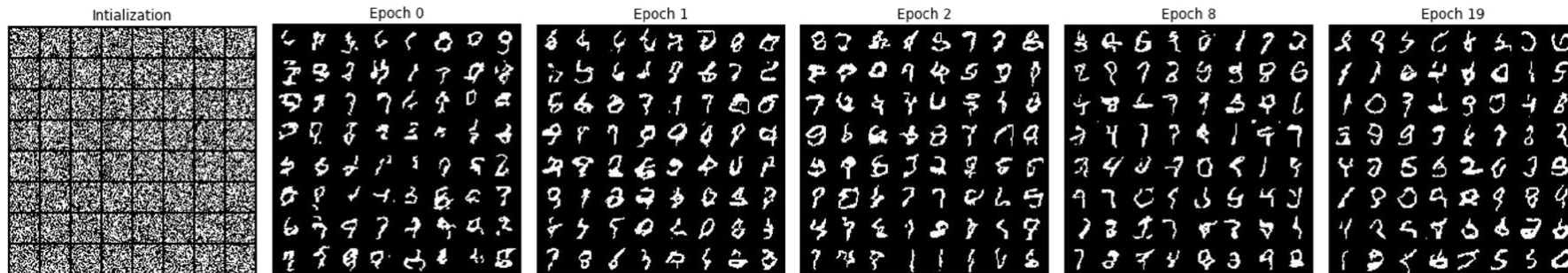
lead



Location Appended Wavenet on MNIST



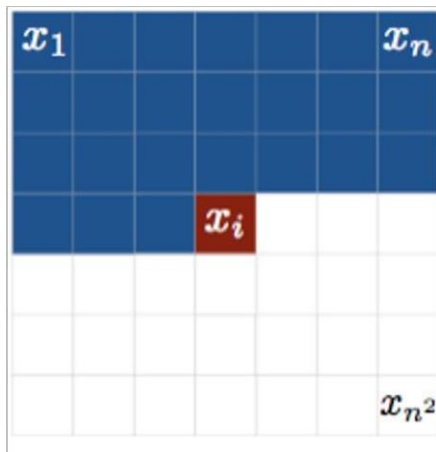
- Append (x,y) coordinates of pixel in the image as input to WaveNet



Masked Spatial (2D) Convolution - PixelCNN



- Images can be flattened into 1D vectors, but they are fundamentally 2D
- We can use a masked variant of ConvNet to exploit this knowledge
- First, we impose an autoregressive ordering on 2D images:

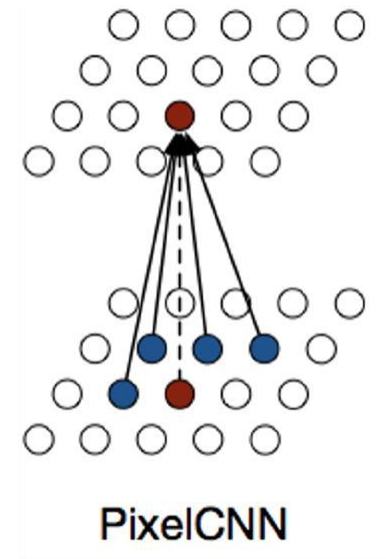
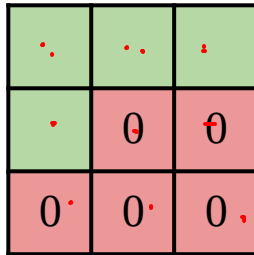


This is called raster scan ordering.
(Different orderings are possible,
more on this later)

<https://neuroverse0.wordpress.com/2020/08/11/pixelrnn-gated-pixelcnn-and-pixelcnn/>

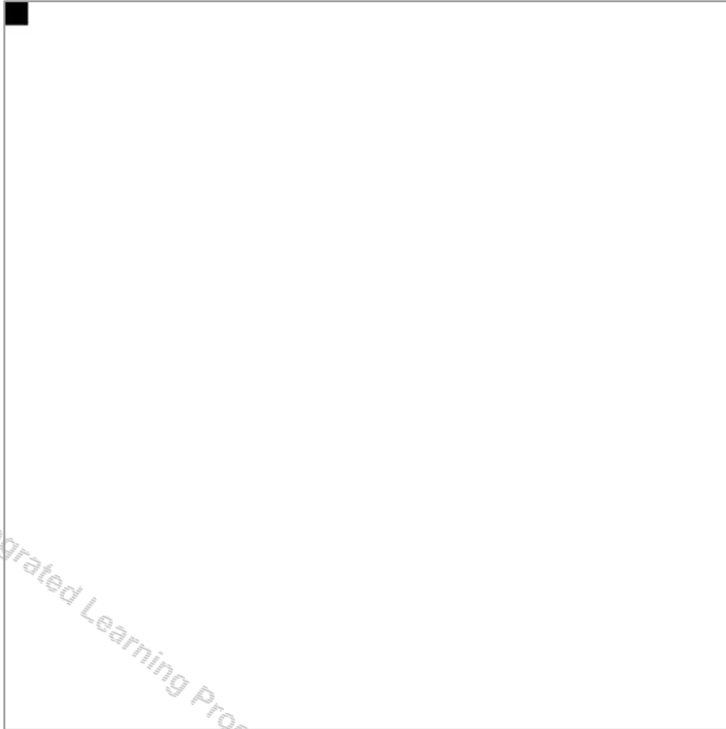
PixelCNN

- Design question: how to design a masking method to obey that ordering?
- One possibility: PixelCNN (2016)



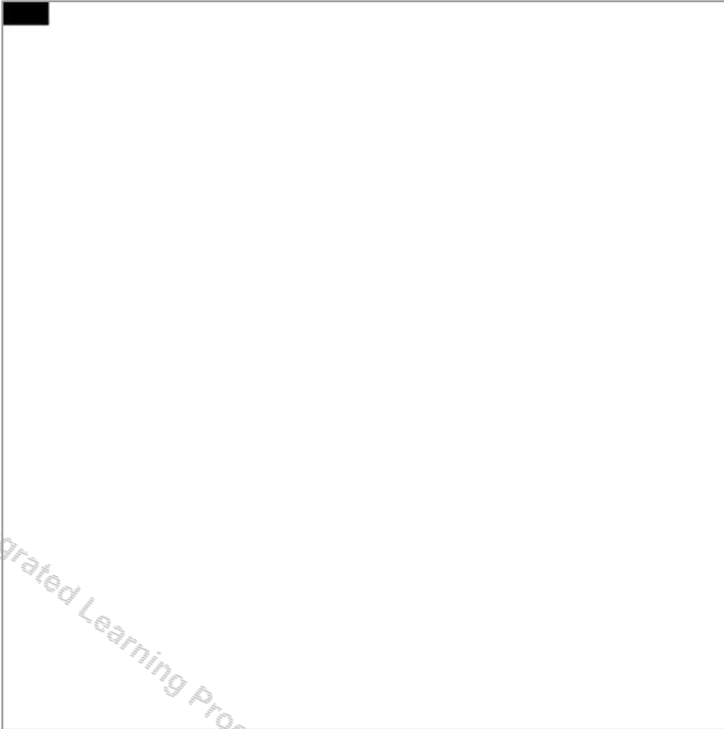
- <https://sergeiturukin.com/2017/02/22/pixelcnn.html>
- <https://medium.com/data-science-in-your-pocket/generative-modeling-using-pixelcnn-with-codes-explained-387c95405651>

Softmax Sampling



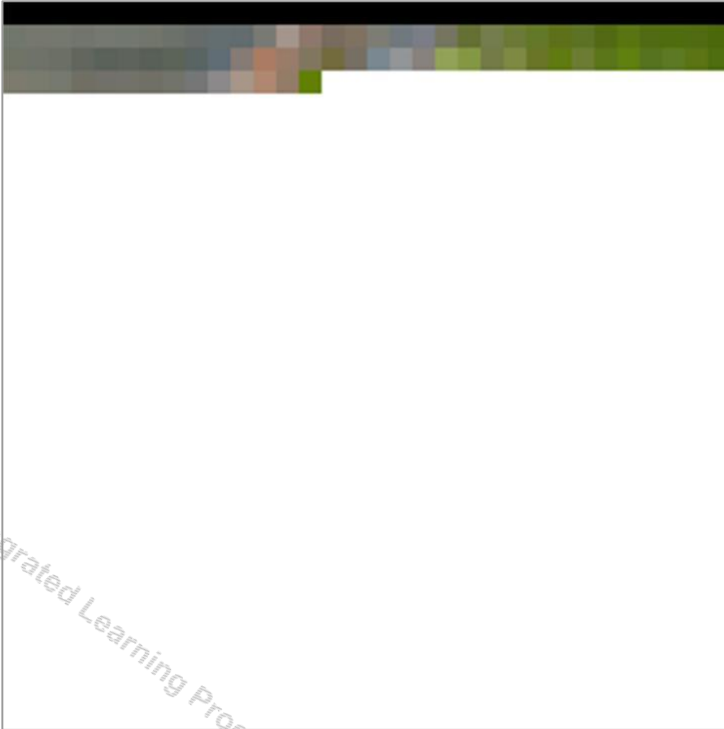
Slide by Aaron van den Oord

Softmax Sampling



Slide by Aaron van den Oord

Softmax Sampling

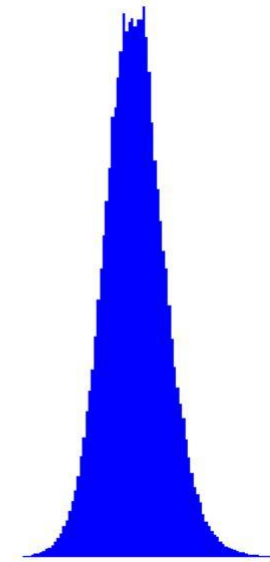


Slide by Aaron van den Oord

Softmax Sampling



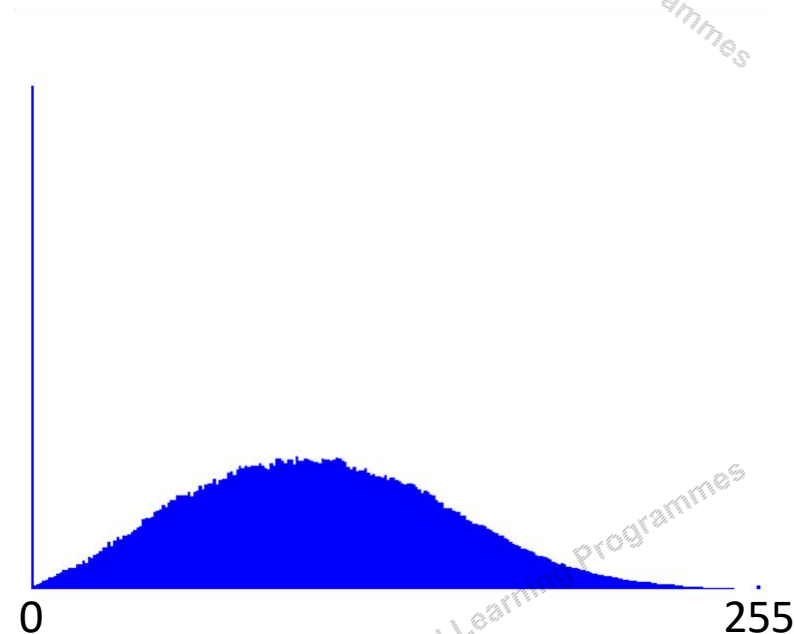
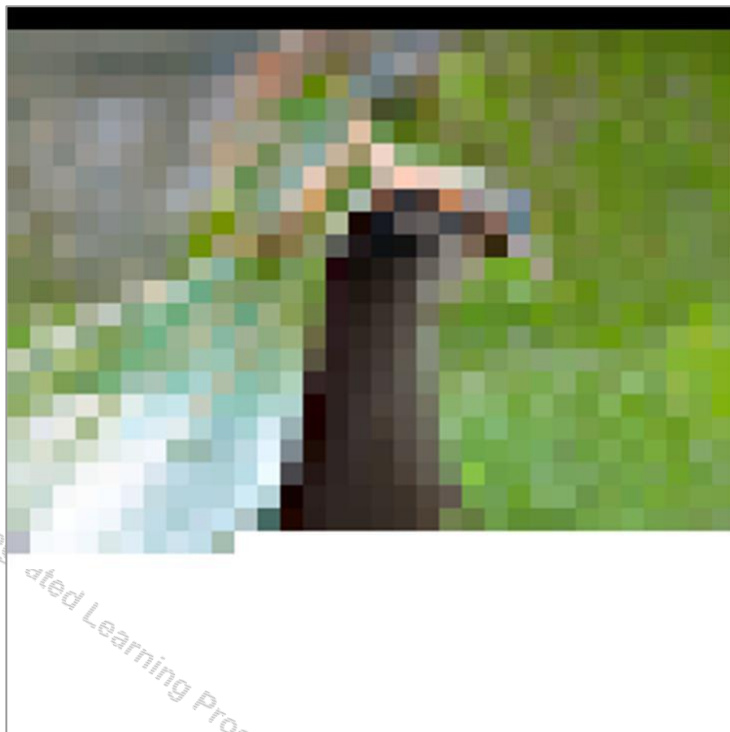
0



255

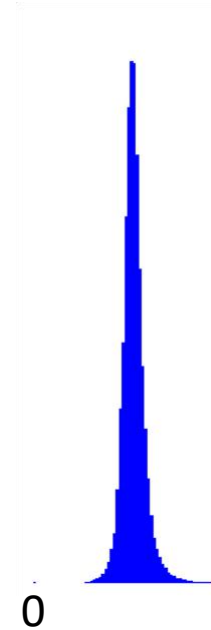
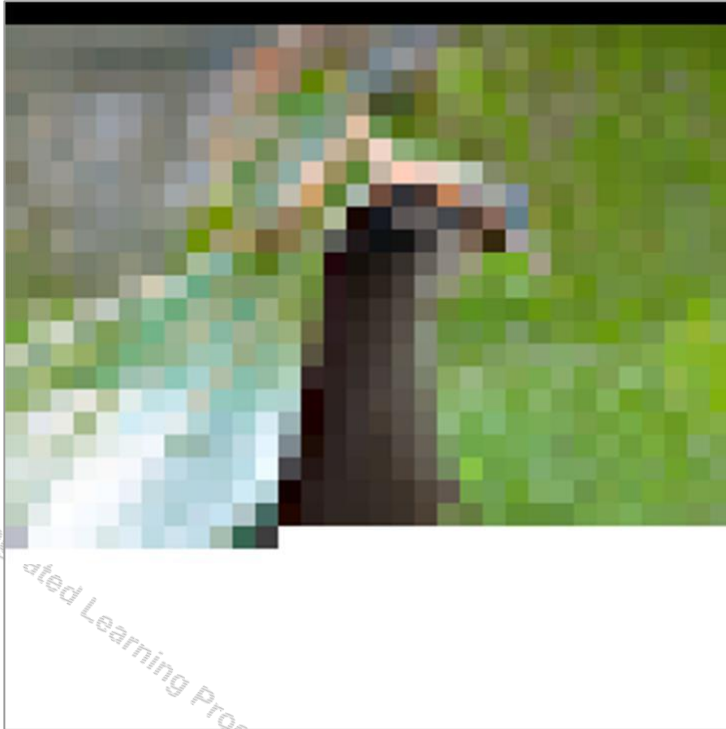
Slide by Aaron van den Oord

Softmax Sampling



Slide by Aaron van den Oord

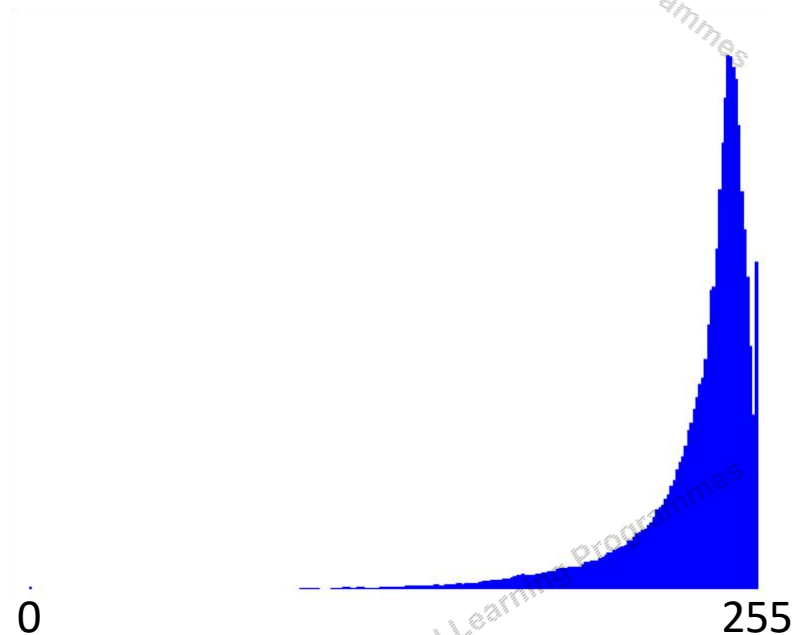
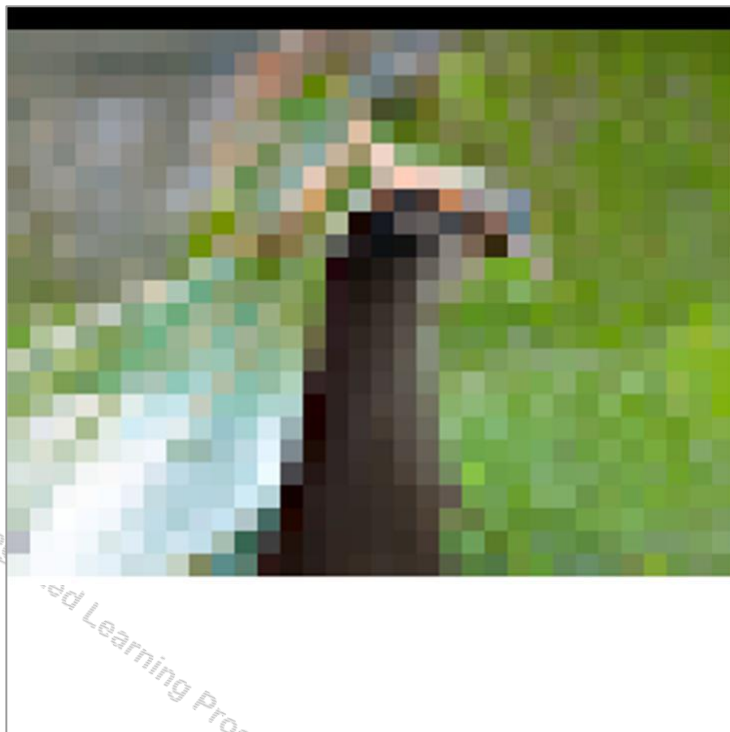
Softmax Sampling



255

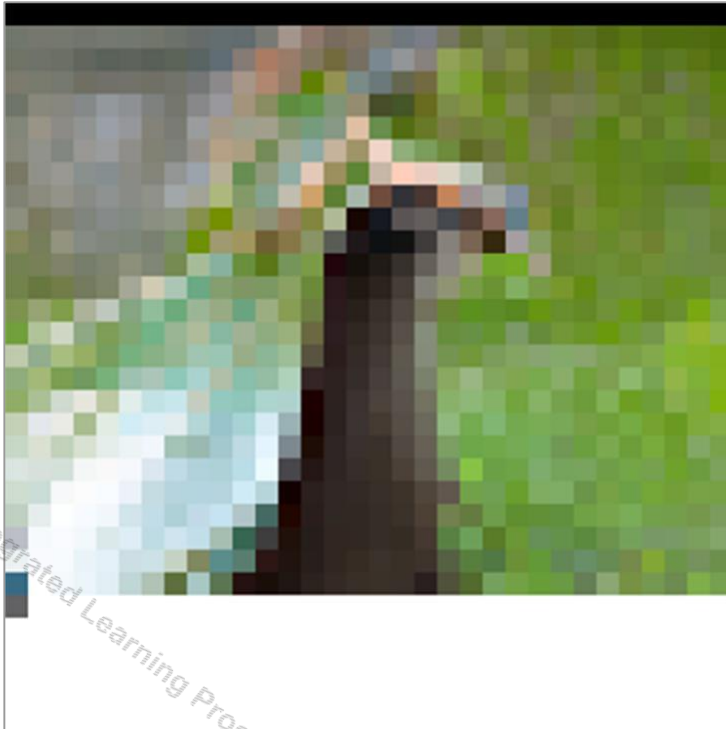
Slide by Aaron van den Oord

Softmax Sampling



Slide by Aaron van den Oord

Softmax Sampling

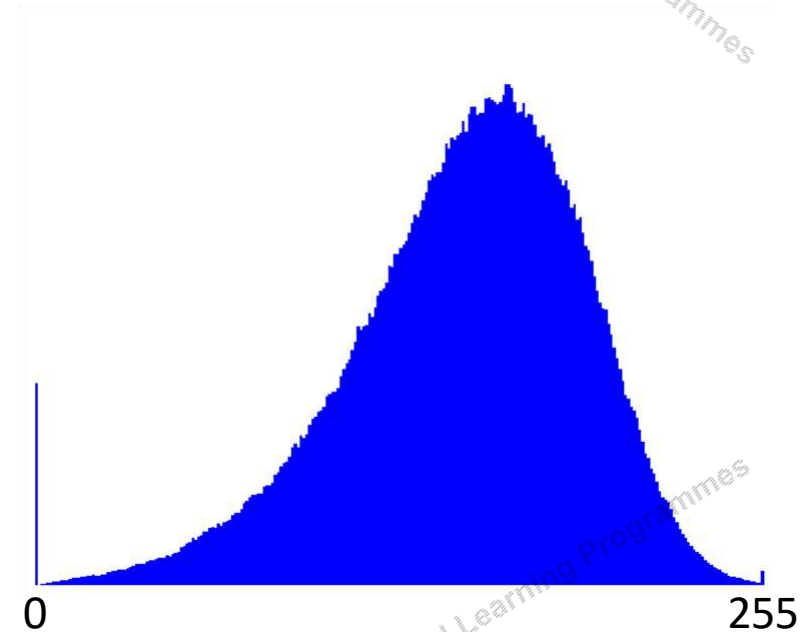
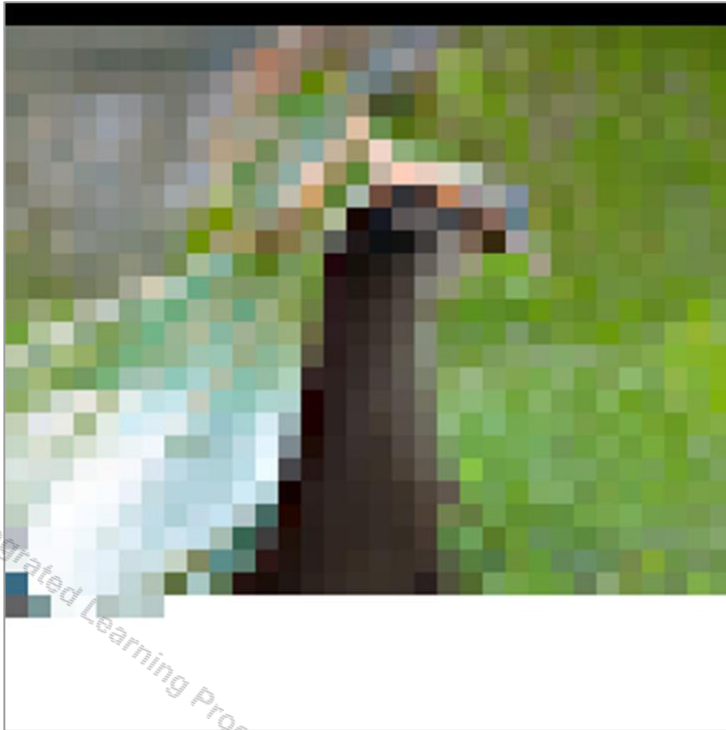


0

255

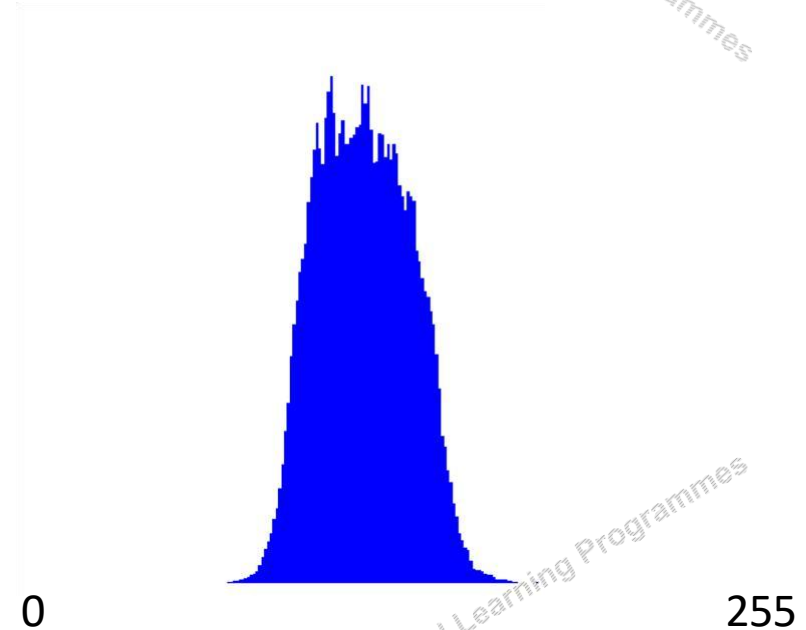
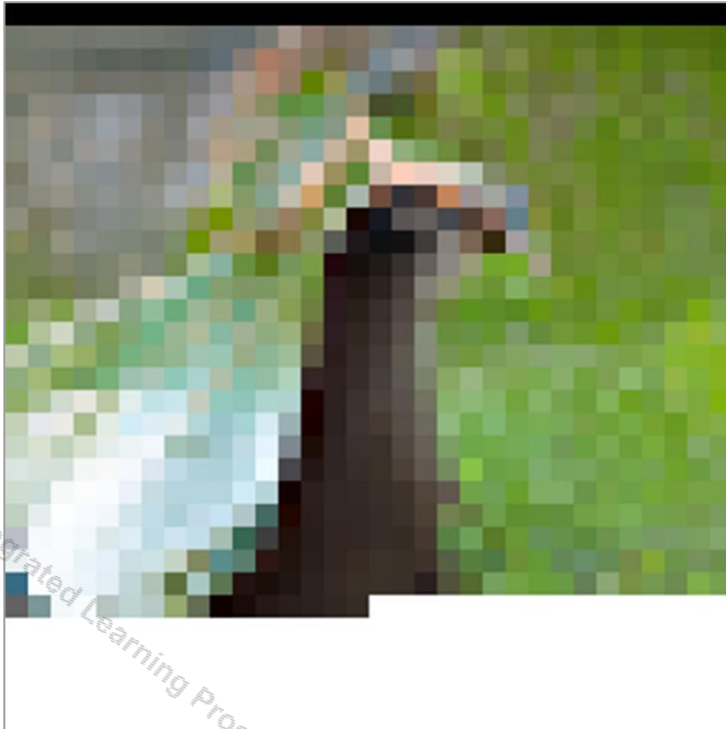
Slide by Aaron van den Oord

Softmax Sampling



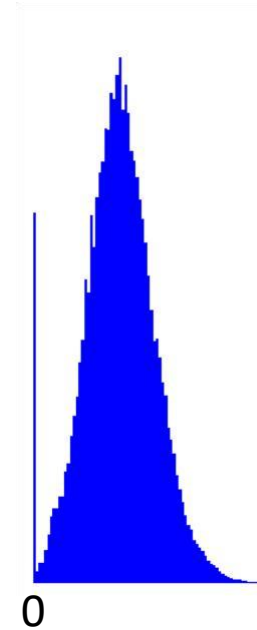
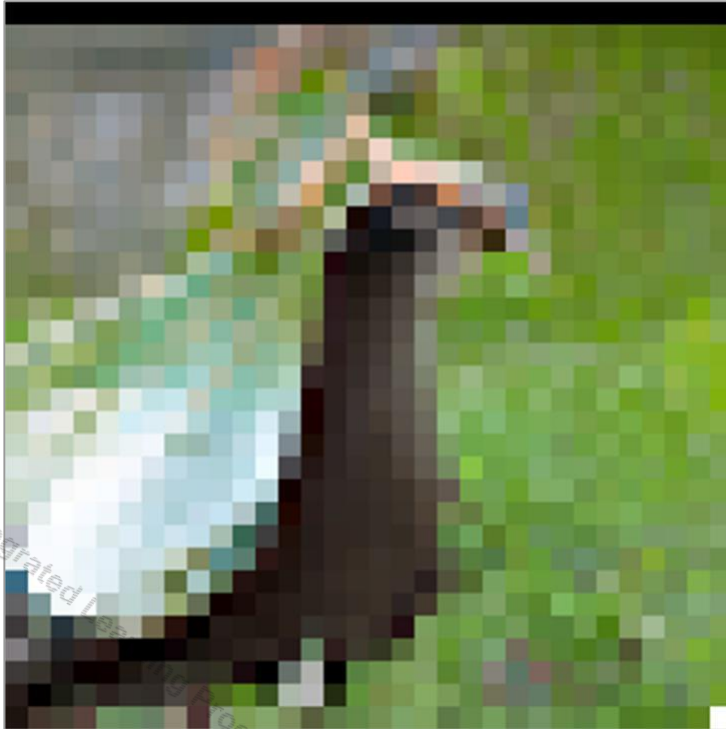
Slide by Aaron van den Oord

Softmax Sampling



Slide by Aaron van den Oord

Softmax Sampling



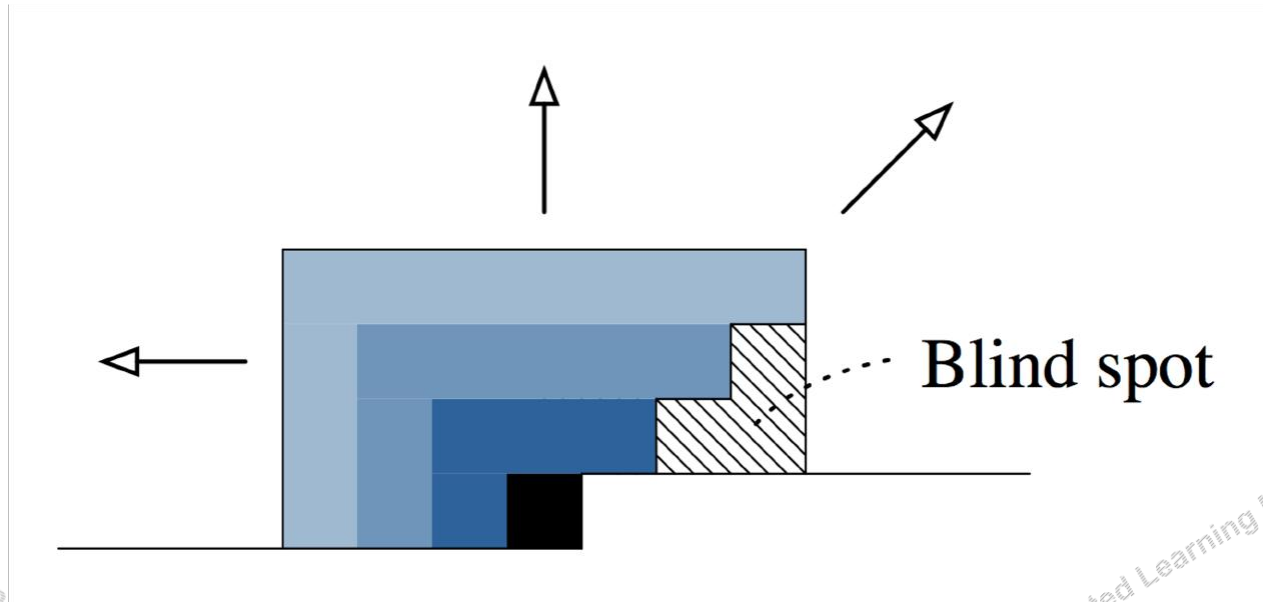
255

Slide by Aaron van den Oord

PixelCNN



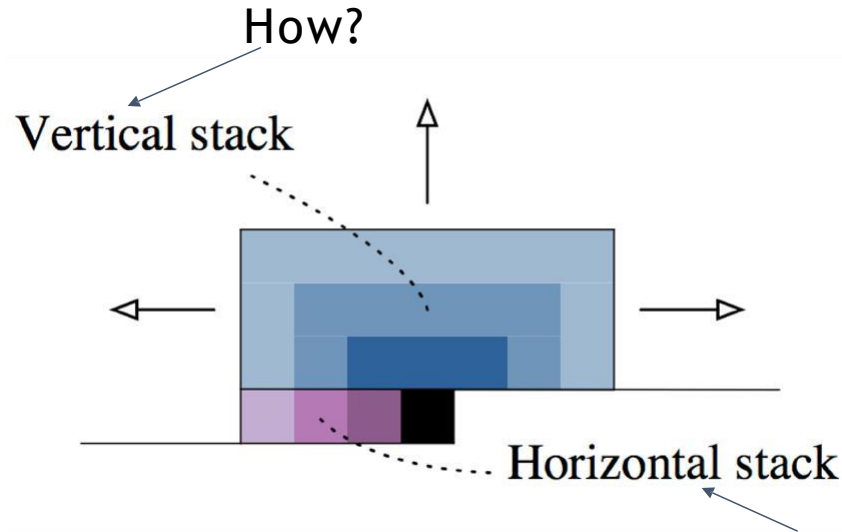
- PixelCNN-style masking has one problem: blind spot in receptive field



- <https://towardsdatascience.com/pixelcnns-blind-spot-84e19a3797b9>

Gated PixelCNN

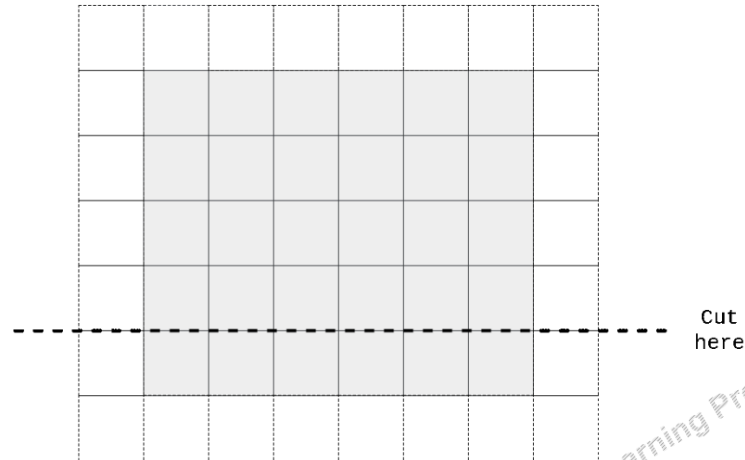
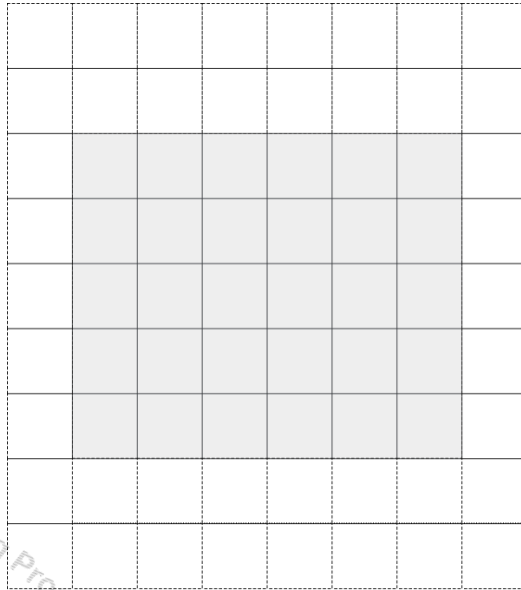
- Gated PixelCNN (2016) introduced a fix by combining two streams of convolutions



This is easy, we know how to do 1D masked conv

Gated PixelCNN

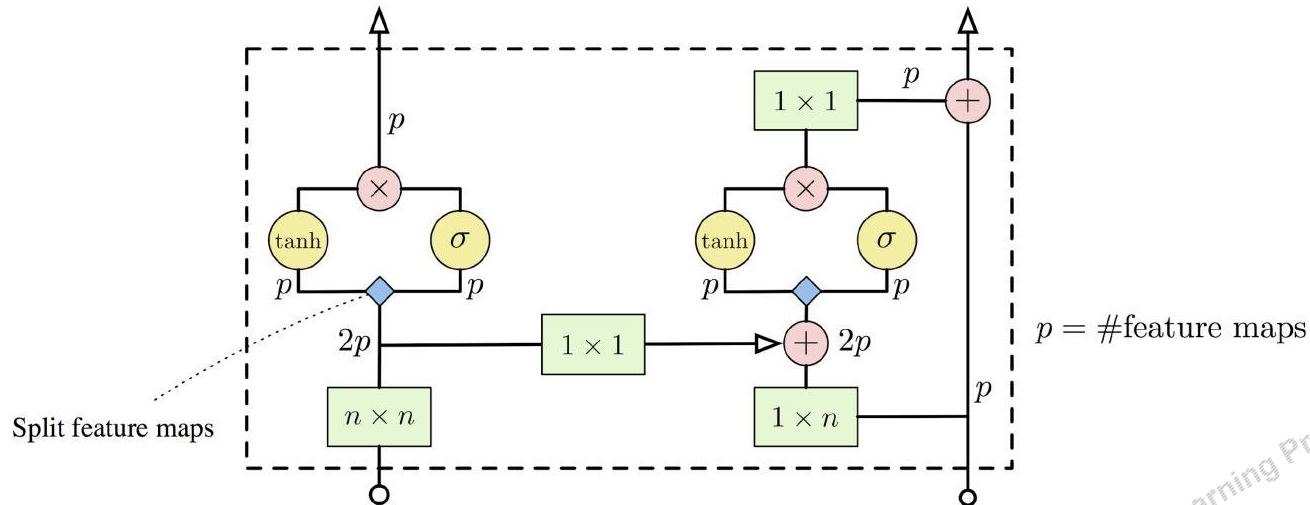
- Vertical stack: through padding, activations at i th row only depend on input before i th row



Gated PixelCNN

- Improved ConvNet architecture: Gated ResNet Block

$$\mathbf{y} = \tanh(W_{k,f} * \mathbf{x}) \odot \sigma(W_{k,g} * \mathbf{x})$$



<https://thomasjubb.blog/autoregressive-generative-models-in-depth-part-3/>

Gated PixelCNN



- Better receptive field + more expressive architecture = better performance

Model	NLL Test (Train)
Uniform Distribution: [30]	8.00
Multivariate Gaussian: [30]	4.70
PixelCNN: [30]	3.14 (3.08)
PixelRNN: [30]	3.00 (2.93)
Gated PixelCNN:	3.03 (2.90)

Table 1: Test set performance of different models on CIFAR-10 : (lower is better), training performance in brackets.

PixelCNN++



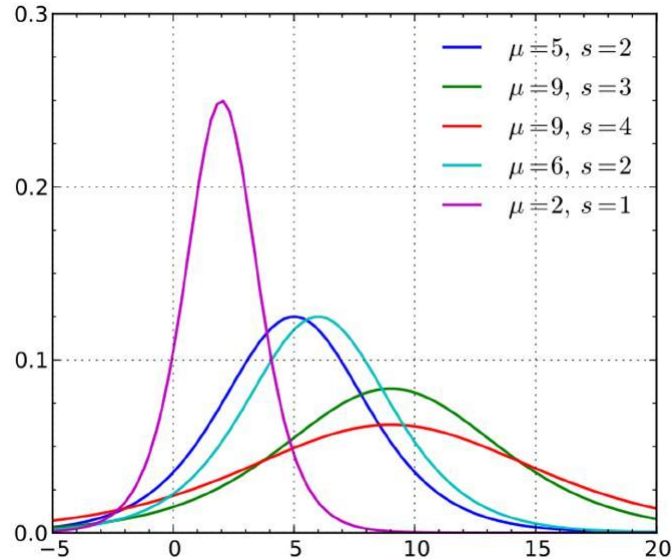
- Moving away from softmax: we know nearby pixel values are likely to co-occur!

$$\nu \sim \sum_{i=1}^K \pi_i \text{logistic}(\mu_i, s_i)$$

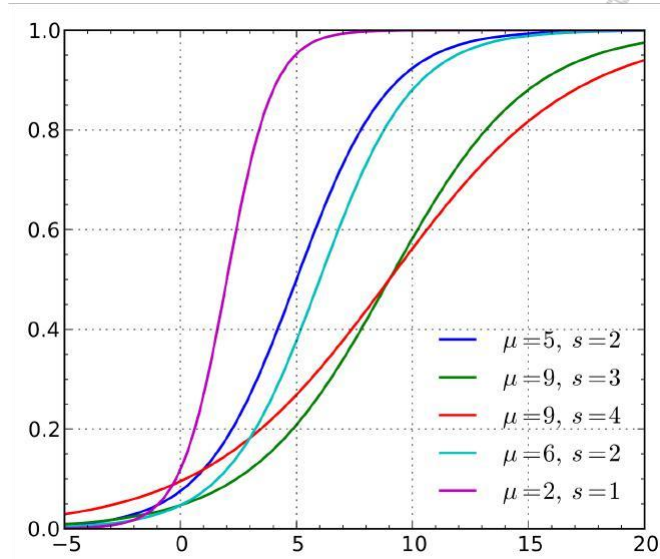
$$P(x|\pi, \mu, s) = \sum_{i=1}^K \pi_i [\sigma((x + 0.5 - \mu_i)/s_i) - \sigma((x - 0.5 - \mu_i)/s_i)],$$

<https://neuroverse0.wordpress.com/2020/08/11/pixelrnn-gated-pixelcnn-and-pixelcnn/>

Recap: Logistic distribution



pdf



cdf

= $\text{sigmoid}((x - \mu) / \text{scale})$

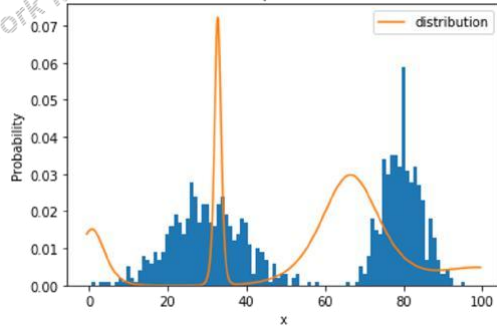
Ex. Training Mixture of Logistics

innovate

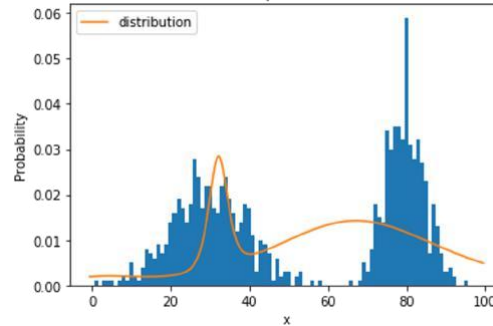
achieve

lead

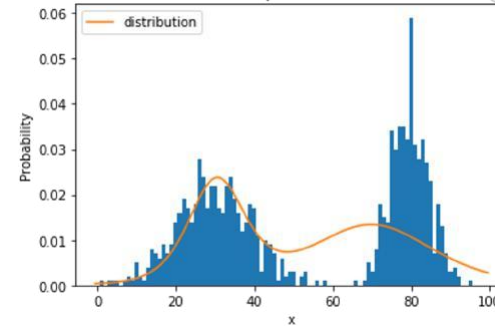
Epoch 0



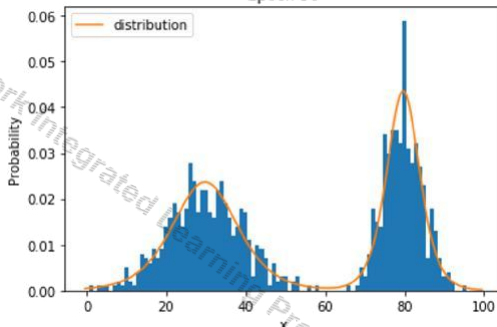
Epoch 2



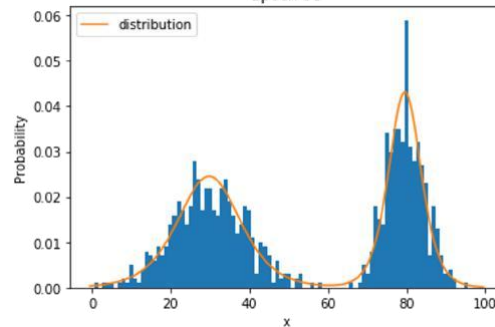
Epoch 10



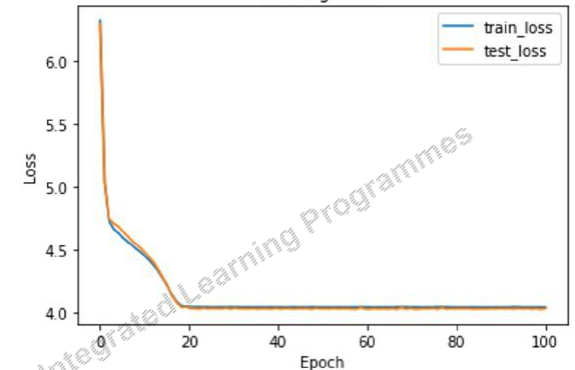
Epoch 50



Epoch 99

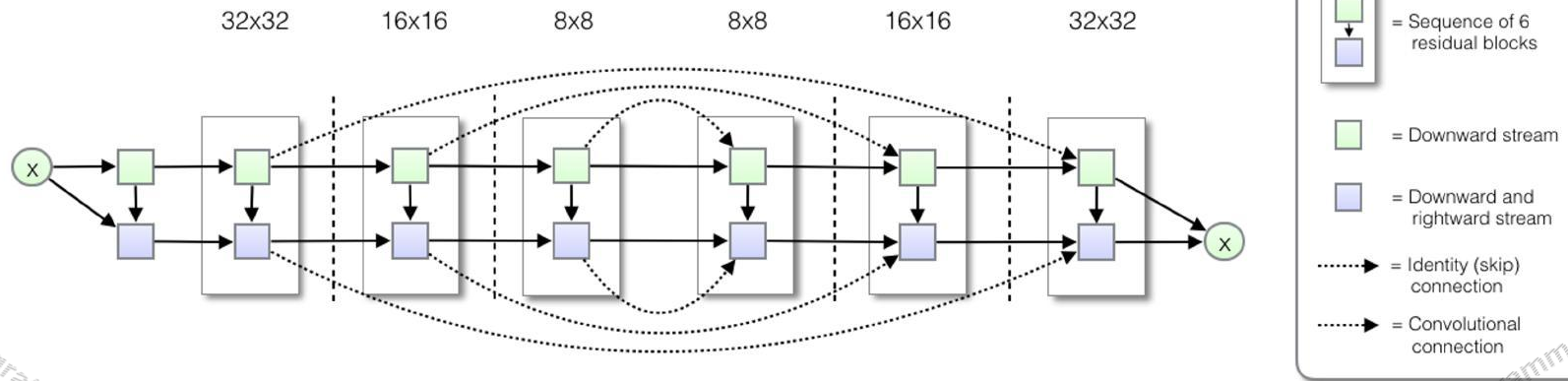


Training Curve



PixelCNN++

- Capture long dependencies efficiently by downsampling



PixelCNN++

innovate

achieve

lead

Model	Bits per sub-pixel
PixelCNN (van den Oord et al., 2016b)	3.14
Gated PixelCNN (van den Oord et al., 2016c)	3.03
PixelRNN (van den Oord et al., 2016b)	3.00
PixelCNN++	2.92

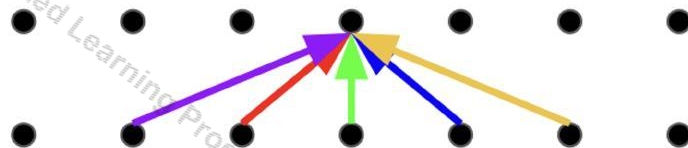
Table 1: Negative log-likelihood for generative models on CIFAR-10

Masked Attention

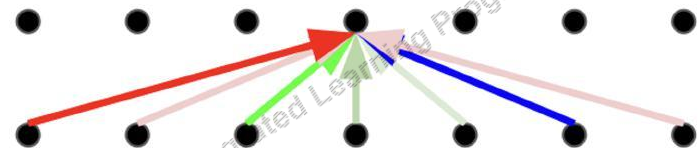


- A recurring problem for convolution: limited receptive field → hard to capture long-range dependencies
- (Self-)Attention: an alternative that has
 - unlimited receptive field!!
 - also $O(1)$ parameter scaling w.r.t. data dimension
 - parallelized computation (versus RNN)

Convolution

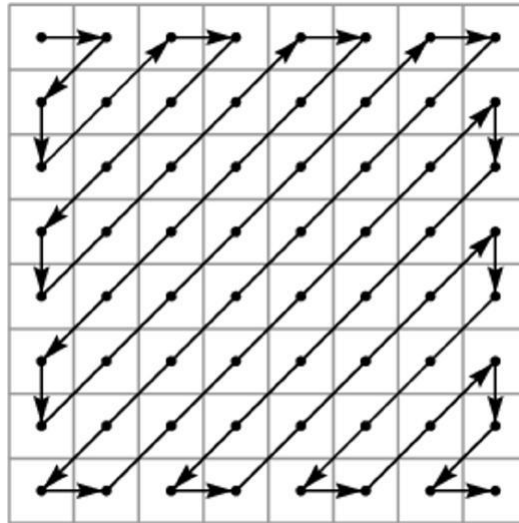


Self-attention



Masked Attention

- Masking enforces causality
- Much more flexible than masked convolution. We can design any autoregressive ordering we want

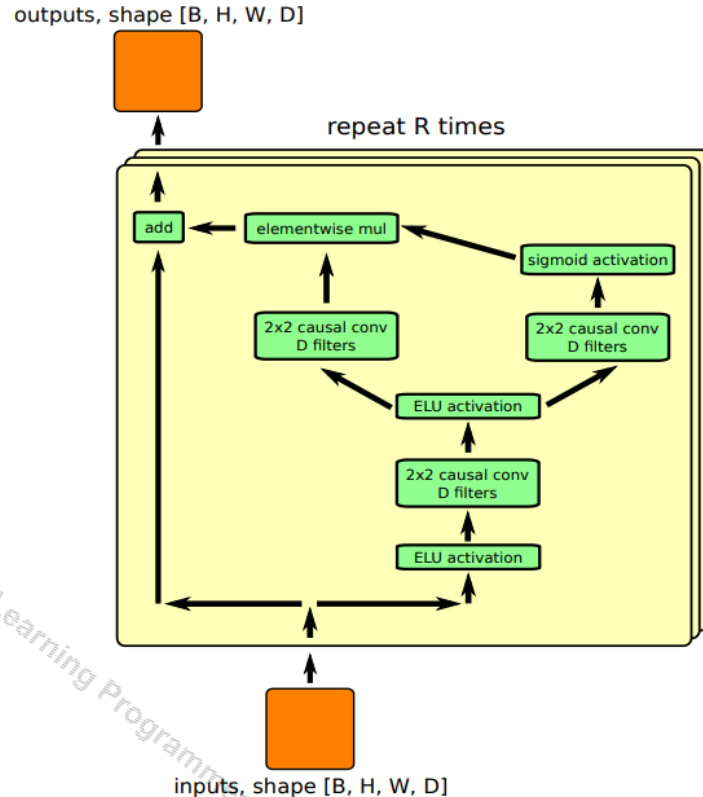


Zigzag ordering

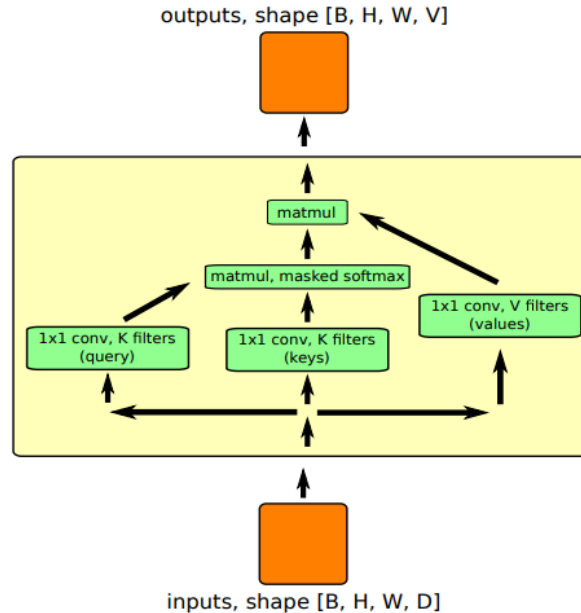
- How to implement with masked conv?
- Trivial to do with masked attention!

Building Blocks in PixelSNAIL

(a) Residual Block (D filters, R repeats)



(b) Attention Block (key size K, value size V)

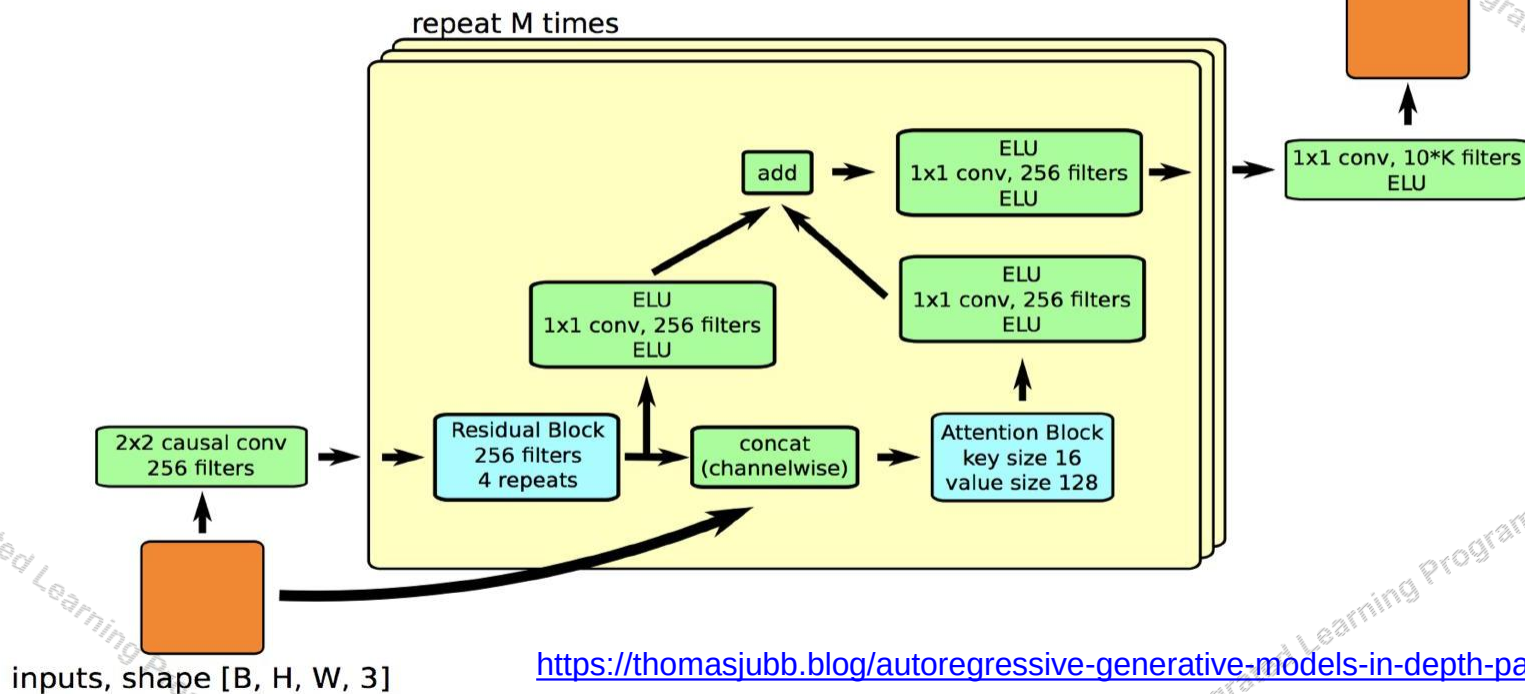


Masked Attention + Convolution



PixelSNAIL: M blocks, K mixture components

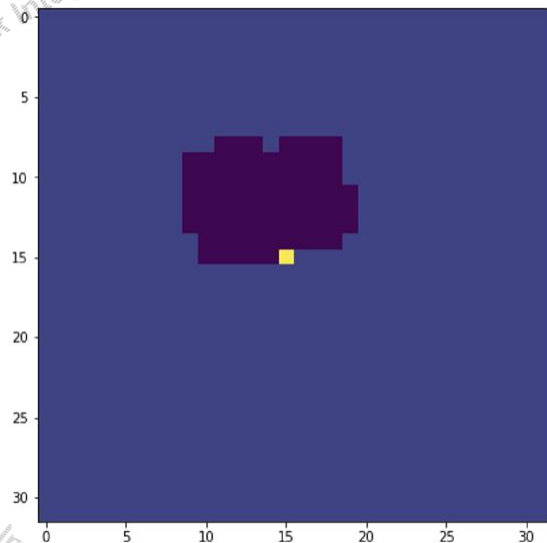
outputs, shape $[B, H, W, 10 \times K]$



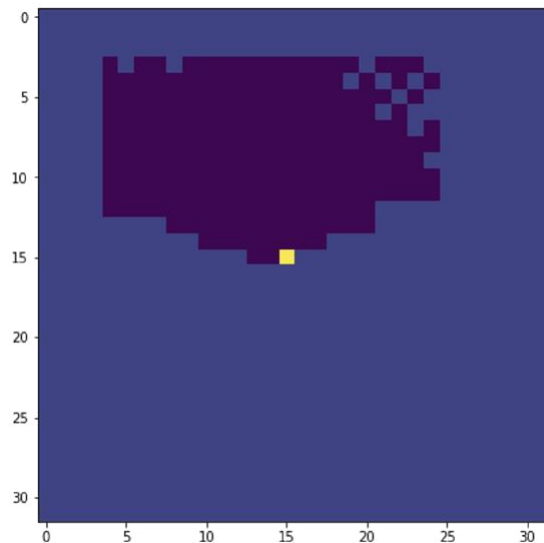
<https://thomasjubb.blog/autoregressive-generative-models-in-depth-part-4/>

<https://thomasjubb.blog/autoregressive-generative-models-in-depth-part-5/>

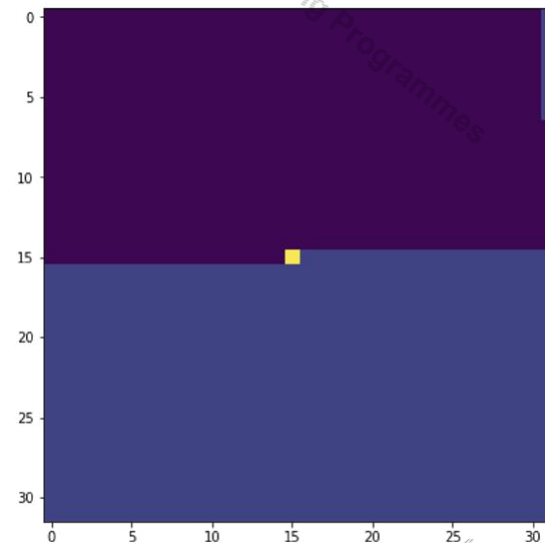
Masked Attention + Convolution



Gated PixelCNN



PixelCNN++



PixelSNAIL

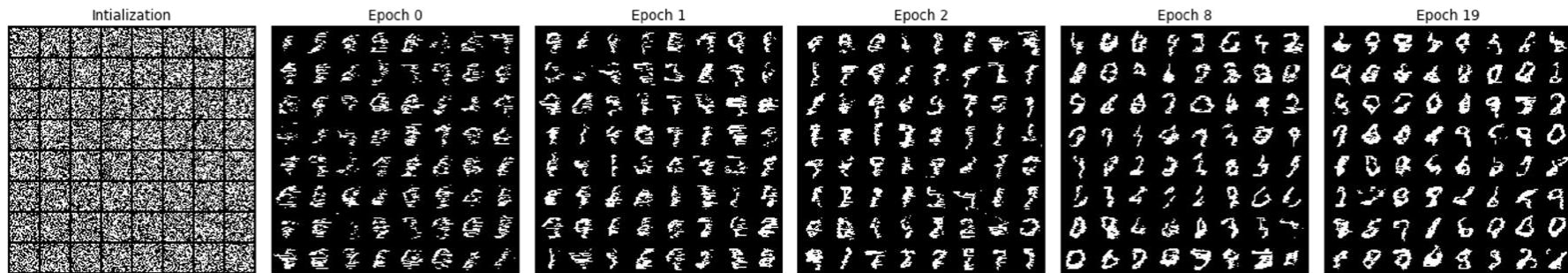
<https://thomasjubb.blog/autoregressive-generative-models-in-depth-part-4/>

Multi-Head Self-Attention on MNIST

innovate

achieve

lead



Masked Attention + Convolution



Method	CIFAR-10
PixelRNN (Oord et al., 2016b)	3.00
Gated PixelCNN (van den Oord et al., 2016b)	3.03
PixelCNN++ (Salimans et al., 2017)	2.92
PixelSNAIL	2.85

Class-Conditional PixelCNN

innovate

achieve

lead



How to condition?

IN: One-hot encoding of the labels

THEN: multiplying by different learned weight matrices in each convolutional layer, and added as a bias channel-wise and broadcasted spatially

$$\mathbf{y} = \tanh(W_{k,f} * \mathbf{x} + V_{k,f}^T \mathbf{h}) \odot \sigma(W_{k,g} * \mathbf{x} + V_{k,g}^T \mathbf{h})$$

<https://arxiv.org/pdf/1606.05328>

Neural autoregressive models: The good



Best in class modelling performance:

- expressivity - autoregressive factorization is general
- generalization - meaningful parameter sharing has good inductive bias

→ State of the art models on multiple datasets, modalities

Masked autoregressive models:

The bad

- Sampling each pixel = 1 forward pass!
- 11 minutes to generate 16 32-by-32 images on a Tesla K40 GPU



Image Super-Resolution with PixelCNN



- A PixelCNN is conditioned on 7×7 subsampled MNIST images to generate the corresponding 28×28 image

Hierarchy: Grayscale PixelCNN



- Design an autoregressive model architecture that takes advantage of the structure of data
- Learn a PixelCNN on binary images, and a PixelCNN conditioned on binary images to generate colored images

Pixel Recursive Super Resolution

innovate

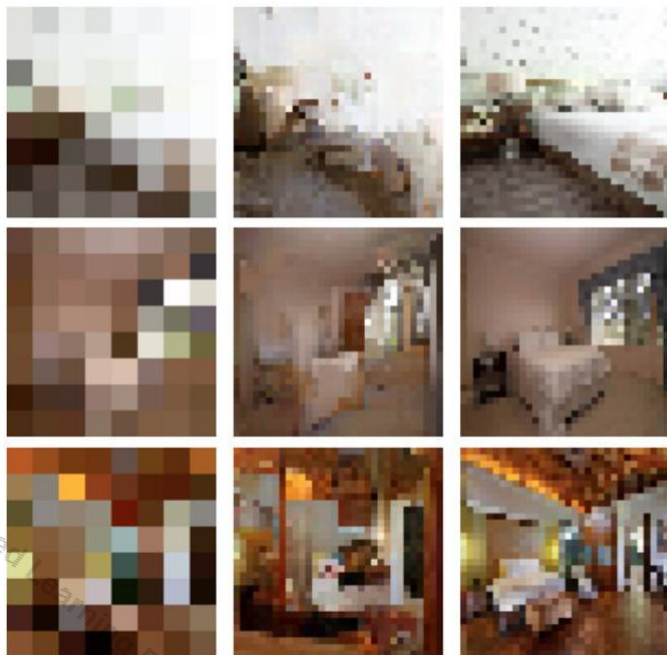
achieve

lead

8×8 input

32×32 samples

ground truth



8×8 input

32×32 samples

ground truth

